

Article

Prediction-Driven Non-Preemptive Scheduling with Bounded Overtaking for Shared Execution Services under Deadline-Driven Bursty Load

Yazhou Guo ^{1,*}, Zexin Lin ², Chengyang Huo ³ and Yurong Song ¹¹ College of Automation, Nanjing University of Posts and Telecommunications, Nanjing 210023, China; gyz@njupt.edu.cn (Y.G.); songyr@njupt.edu.cn (Y.S.)² College of Artificial Intelligence, Nanjing University of Posts and Telecommunications, Nanjing 210023, China; b25060222@njupt.edu.cn (Z.L.)³ Division of Logistics and Transportation, Shenzhen International Graduate School, Tsinghua University, Shenzhen 518055, China; huocy24@mails.tsinghua.edu.cn (C.H.)

* Correspondence: gyz@njupt.edu.cn

Abstract

Services that grade students' programs, run cloud functions or test software run many users' jobs on k shared servers without interruption, each under a time limit L , with bursty arrivals. Ordering the queue by predicted running time shortens most waits, but it can hold a long job back for a long time, and a wrong prediction can do the same to any job. What can such a service promise every job when predictions may be arbitrarily wrong? Under any rule that never idles a server while jobs wait and never interrupts a job, a job's extra wait relative to first-come first-served (FCFS) equals the net running time of the jobs that overtook it, divided by k , up to an error of $(2 - 2/k)L$ that cannot be improved. Limiting that running time is therefore sufficient for a per-job promise and, up to an additive $O(kL)$, necessary. A guard enforcing such a limit bounds every job's wait against FCFS for any arrivals, predictions and underlying policy, with exact constant $(3 - 2/k)L$; if completion reports arrive up to δ late, the bound grows by $L + \delta$. On a university grading log at peak load, scoring each job only from results already returned to its own class at its arrival, a guard promising 600 seconds keeps 0.705 of the reduction in 99th-percentile wait that shortest-job-first achieves over FCFS, and 0.894 on three terms held back until the method was frozen.

Keywords: non-preemptive scheduling; scheduling with predictions; multiserver queues; bounded overtaking; per-job guarantee; first-come first-served; runtime prediction; cluster scheduling; heterogeneous graph neural network

MSC: 90B35; 68M20; 60K25; 68W27Academic Editor: Firstname
Lastname

Received: date

Revised: date

Accepted: date

Published: date

Citation: To be added by editorial staff
during production.**Copyright:** © 2026 by the authors.

Submitted for possible open access

publication under the terms and

conditions of the Creative Commons

Attribution (CC BY) license

<https://creativecommons.org/licenses/by/4.0/>.

1. Introduction

Many computing services let a large number of users share a fixed set of machines. A university system that compiles and tests every programming assignment its students submit works this way [1]; so does a service that runs a software project's tests each time a developer changes the code, known as continuous integration (CI), and a cloud platform that runs small user functions on request. Each submission becomes a job, and the jobs wait in one queue for k identical machines, which we call servers. A server that becomes

free takes the next job and runs it without interruption, until the job finishes or reaches a time limit L that the service imposes on every job, and then reports the completion to the scheduler over the network.

Two features of the demand make such a service slowest exactly when it is busiest. Submissions follow external calendars, such as a coursework deadline or a release date, so they arrive in bursts, and the crowding before a fixed cutoff has been measured directly [2–4]. A small fraction of the jobs also carries most of the work. In the log of cloud-function calls we use [5,6], the longest 1% of the calls account for 44% of the total running time. In the two logs of students' programming submissions, with each running time capped at that platform's time limit, the longest 1% account for 37.5% and 27.5%. The university grading system shows both features at their sharpest, and it is our first case. Before a common deadline, submissions that looped until their time limit occupied the graders, and the waits of everything queued behind them grew in a cascade [7]. The manual of a programming-contest judging system recommends one judging machine per 20 teams and warns that waits grow with fewer machines [8].

Ordering the queue by predicted cost exploits the second property, as a queue of three jobs on one server shows (Figure 1). Job A loops and is stopped by the 60-second limit; jobs B and C take 0.2 seconds each. In arrival order the mean wait is 40.1 seconds; shortest first, it is 0.2 seconds, at a cost of 0.4 seconds to A . These numbers are arithmetic and rest on one assumption, that the scheduler knows before A starts that A will be long. The ordering has a second cost, which does not need a wrong estimate: a long job waits while shorter ones keep arriving ahead of it. In our simulation of the busiest period we tested, driven by the recorded submissions, shortest-job-first with exact running times kept one job waiting 4,755.5 s, 79.3 time limits, longer than it would have under first-come first-served (FCFS) order. With predicted costs in place of exact ones, the worst such delay was 5,904.5 s, 98.4 time limits (Section 8.4).

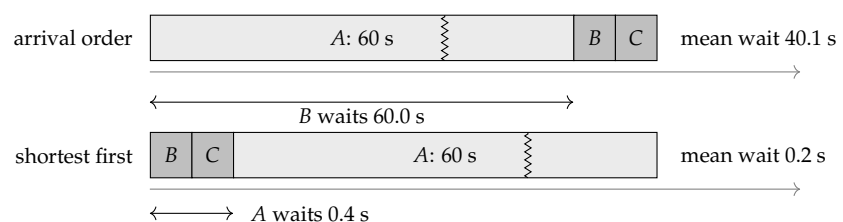


Figure 1. The three-job example of Section 1 on one server: A is stopped at the per-job limit $L = 60$ s, B and C take 0.2 s each, and all three are present at $t = 0$. The boxes are schematic, not to scale, and the zigzag marks the elided part of A . The two orders execute the same total work; reordering moves 60 seconds of waiting off B and C and puts 0.4 seconds onto A .

A line of work on algorithms with predictions bounds how an algorithm performs as a function of the error of the predictions it is given [9,10]. For scheduling, these bounds compare the total or mean completion time with that of an optimal schedule, and the constant-factor results assume that the prediction error is bounded [11–15]. A predictor in service carries no such bound, and an operator is asked about individual jobs rather than averages. Two results come closer: one bounds the worst slowdown of any job on a machine that may interrupt jobs [16], and one bounds a request's delay in a queue with steady, light-tailed demand [17,18]. We have not found a guarantee that is stated for every job against FCFS, holds for any arrival sequence and any predictions, allows no interruption, covers k servers, and can be enforced while any scheduling rule runs underneath. This paper answers three questions.

1. Can a job's running time be estimated when it arrives, from the history available at that moment?

2. How should a scheduler that cannot interrupt jobs order the queue once it has such estimates?
3. What can the service promise every job when the estimates are wrong, and when the completion reports it relies on arrive late?

For the third question we change the reference point instead of assuming more about the predictions. For each job, we bound how much longer it waits than it would have under FCFS on the same sequence of submissions, with no assumption about the arrival process or about prediction quality. The mechanism is an overtake budget. For every waiting job the scheduler adds up the running time of the jobs that arrived later but were started ahead of it, and as soon as that total reaches the job's budget, the job is served before any later arrival. This guard sits on top of any scheduling rule, including one driven by a learned predictor, and changes only which job starts next; a cap $B_{\max}$ on the budget fixes the promise. The budget counts running time rather than the number of jobs that overtake, because of an identity we prove: under any scheduling rule that never leaves a server idle while jobs wait and never interrupts a job, a job's extra wait relative to FCFS equals the net running time of the jobs that overtook it, divided by k , up to an error of at most $(2 - 2/k)L$. The guard learns of completions from the servers' reports, so we also bound what it promises when these reports cross a network and arrive late.

For the first two questions we compare three kinds of predictor, trained and tested on the same data under the same rule about what may be seen when a job arrives: gradient-boosted decision trees [19], a gated recurrent unit (GRU) that reads each user's earlier jobs, and a heterogeneous graph neural network (GNN) over users, programming tasks and assignments, of the relational graph convolutional network (R-GCN) type [20]. The history available at arrival separates the long jobs from the short ones on all three platforms we test it on. The graph network's Spearman correlation lies between those of the two tree models, its AUROC is within 0.005 of both, and its edges add nothing measurable over the same network without edges, so the information lies in each user's and each task's own history. On tasks with fewer than five earlier results the graph network ranks jobs better than the trees, by 0.146 in Spearman correlation, but the same network without edges does nearly as well, 0.142 (Section 4). The scheduler orders jobs by their expected running time given the observed features, and for our fitted predictor, treating a long job as short costs about ten times more than treating a short job as long (Section 8.3).

This study makes three contributions.

1. We prove the identity above and show that its error constant $(2 - 2/k)L$ cannot be improved. Conversely, any per-job promise of this kind limits the running time of the jobs that overtake a job, up to an additive $O(kL)$, whereas a limit on how many jobs overtake it is not required and, on our main log, is looser by a factor of 150.
2. We give the guard and prove that it keeps every job's extra wait over FCFS below $B_{\max}/k + (3 - 2/k)L$, for k servers, any arrivals and any predictions; the constant cannot be improved. An operator who wants to promise G seconds sets $B_{\max} = k(G - (3 - 2/k)L)$ and needs no data to do so. The same promise keeps the largest flow time below $(3 - 2/k)$ times its optimum plus G . When completion reports arrive up to δ late, the promise grows by $L + \delta$. It does not grow if each server reports its own completion before taking its next job and dispatch takes no time, and replicas of the scheduler that hold the same reports agree on which jobs the guard must serve next. We also bound the absolute wait from the arrival record alone, and compare the guard with a maximum waiting time, which makes a promise of the same form.
3. We evaluate the method on six logs: two from university systems that grade students' programming submissions [21,22], one from a cloud-function platform [5], two from CI machine pools [23,24] and one from an industrial compute farm [25,26]. Every

simulated job is checked against the proved bound. At the busiest load, with every score computed from the results available when its job arrives, a guard that promises 600 seconds keeps 0.705 (bootstrap interval [0.648, 0.768]) of the reduction in the 99th-percentile wait before deadlines that shortest job first (SJF), which knows every running time in advance, achieves over FCFS (Section 8.1).

Two qualifications apply throughout. The grading logs record how long each submission ran but not how long it waited, so the shared queue in our simulations is our own construction (Section 7). Most numbers reported come from data we used while developing the method. Three academic terms of the main log were held back unread and opened once, after the method and its parameters were frozen. On them the guard keeps 0.894 of the reduction at peak load, against 0.705 on the development terms (Section 8.2).

The rest of the paper is organised as follows. Section 2 reviews related work, and Section 3 states the model and the measures of performance. Section 4 compares the predictors, Section 5 describes the scheduler and the guard, and Section 6 proves the identity and the per-job bounds. Section 7 describes the logs, Section 8 reports the experiments and Section 9 discusses the limitations; longer proofs are in the Supplementary Materials.

2. Related Work

2.1. Scheduling with Predictions

Algorithms with predictions are judged by consistency, how well they do when the predictions are exact, and robustness, how well they do when the predictions are arbitrarily wrong [9,10]. In scheduling, predicted sizes have been used to minimise the total completion time of jobs whose sizes are unknown [11], learned machine weights to assign jobs online [12], and a predicted order of the jobs in place of their sizes [13]. Queueing models treat the prediction as a random variable [27], bound its error multiplicatively [14] or measure it as a distortion [28,29], and further work covers many-server limits [30], the cost of obtaining predictions [31] and the worst slowdown of any job on a machine that may interrupt jobs [16]; Mitzenmacher and Shahout survey this line [15]. These results bound a mean or a competitive ratio against an optimal schedule, and the bound degrades with a measure of the prediction error. In their terms, our guarantee is a robustness statement made for each job and on every sample path: it holds for any predictions, but it bounds the loss against FCFS and does not by itself promise a gain, which Section 8 measures.

2.2. Per-Job Guarantees and Bounded Overtaking

Schwiegelshohn and Yahyapour call a schedule λ -fair when later submissions cannot increase a job's flow time by more than a factor λ , and bound fairness and efficiency together for a policy that interrupts whole groups of processors [32]. Batch schedulers measure a job's delay against a fair start time, obtained by simulating the schedule again without the later arrivals [33,34]. Conservative backfilling gives every queued job a reservation that later jobs cannot delay [35,36], slack-based backfilling lets later jobs delay a job only within a slack fixed at submission [37], and a preemptive variant of the Extensible Argonne Scheduling sYstem (EASY) backfilling [38] kills and restarts the backfilled jobs that would delay a higher-priority one [39]. These guarantees rest on reservations or on interrupting jobs; ours is additive, needs no interruption and is measured against FCFS on the same input.

Bounded overtaking has been studied in two units. The first is a count: Nudge lets a small arriving job swap past the job at the tail of the queue [40], Nudge- K lets it pass up to K others while no job is passed more than once [41], with later refinements [42], and finite-skip models serve only among the oldest n waiting jobs [43]. The second is work: γ -Boost measures the work of later arrivals that are served ahead of a tagged job [17,18].

These analyses assume stationary Poisson arrivals and target means or tail asymptotics, not a cap enforced on every sample path. Our wrapper has the architecture of Load Balancing Guardrails [44], which leaves an arbitrary dispatching policy free except where a constraint must be enforced, although that work optimises mean response time in heavy traffic. Additive bounds against a reference discipline are standard in packet scheduling [45–48], and the coefficient of Graham’s list-scheduling bound [49] has the same shape as the $(2 - 2/k)$ of Section 6. Supplementary Section S9 compares these guarantees in detail.

2.3. Prediction-Driven Scheduling in Deployed Systems

Backfilling schedulers for high-performance computing replace users’ runtime estimates by predictions learned from past jobs, and report mean slowdown or mean wait [50–52]. Closest to our first case, Zhang, Wu and Lu predict the running time of submitted programs in a judging queue and order the jobs by prediction inside fixed-size windows [53], which limits overtaking by position; they report no per-job bound. Servers for large language models, which cannot cheaply interrupt a request, learn a relative order of the requests [54] or target the latency tail, because prediction-driven policies are fragile when the workload shifts [55]; others promote a request once its wait passes a threshold [56] or a multiple of its service time [57]. Serverless platforms approximate shortest remaining time first from user space [58] or learn a priority for each function [59]. None of these states a bound for individual jobs. A maximum waiting time does carry one, of the same form as ours (Section 6.6), and our guard can be placed around any of these schedulers.

2.4. Deadline-Driven Load

People concentrate their actions before a cutoff [2] and register in bursts before a deadline [3,4]. When the arrival rate changes as fast as jobs are served, congestion peaks after the arrivals do [60], and staffing rules need time-varying approximations [61]. These results size the capacity; they give no bound for an individual job once a queue has formed, and we keep the capacity fixed.

2.5. Relational Models for Cost Prediction

A job’s cost may depend on its user, its task and the relations between them, which is the setting of relational deep learning. On the RelBench benchmarks, graph models over linked tables beat gradient-boosted trees fitted on the columns of a single table [62], while on single tables tree ensembles still outperform deep models [63]. We therefore compare a heterogeneous graph neural network over users, tasks and assignments [20,64] with gradient-boosted trees [19] that carry engineered history features. The graph network is judged by its prediction scores only; the trees and a recurrent model are also judged by the waits they produce in the queue.

3. Problem Model

3.1. Jobs, Servers and Dispatch

A finite set of n jobs is served by k identical servers. We say *server* for what a deployment calls an executor, a judging machine or a worker, and *job* for a submission, a function call or a test run. Job i arrives at time $a_i \geq 0$ and needs C_i units of service, which we also call its *cost* or its *work*. Service is non-preemptive. A server that starts a job keeps it until the job finishes or has run for the time limit L , at which point the system stops it. Since C_i is the work actually executed, $0 < C_i \leq L$, and a job stopped at the limit contributes exactly L . For a stopped job, the waiting-time guarantee says nothing about the lost result. A *task* is the programming exercise a job attempts, so many jobs share one task.

Servers are work-conserving, which means that no server is idle while a job waits. Jobs are numbered in arrival order, with ties broken by input order, and we write $j < i$ when j comes first in this order, so $j < i$ implies $a_j \leq a_i$. At a common instant, completions are processed first, then arrivals, then dispatches. A *policy* is any rule that, whenever a server is free and a job waits, picks one waiting job and starts it. Such a rule may be randomised, learned, priority-driven or driven by predictions of any quality. We make no stochastic assumption about the arrival times, the costs or their dependence, because the load we care about follows a calendar and is not stationary.

3.2. What the Scheduler Knows

A job's cost is hidden when it arrives, and the scheduler learns C_i only when i completes. At arrival it sees a feature vector x_i , built from the history of earlier jobs, and a score $\hat{C}_i$ computed from x_i . This history must be causal. Write done_j for the instant at which the result of job j becomes readable, and $d \geq 0$ for the delay before it reaches the store the features are read from. Then

$$\text{information about } j \text{ may enter } x_i \iff \text{done}_j + d \leq a_i. \quad (1)$$

In a simulation driven by a log, done_j can be read in two ways. One of the two ways takes the time at which the result appeared on the original platform, as the log records it. However, in the simulated queue the job completes at a time that depends on the policy being simulated. A feature that is causal on the original platform's clock may therefore use a result that the simulated queue has not produced yet. On the original clock we run the prediction study of Section 4, and on the simulated one the headline scheduling results (Section 8.1). The guarantees of Section 6 hold under either reading, since they assume nothing about the scores.

3.3. Waiting Time, the FCFS Reference and the Service Guarantee

Let s_i^P be the start time of job i under policy P and $W_P[i] = s_i^P - a_i$ its waiting time. FCFS always starts the earliest-arrived waiting job, and when several servers become free at one instant it starts jobs in arrival order. We run FCFS on the same arrivals, costs and k as P , so that the two are compared job by job on one sample path. The *excess* of job i is

$$\text{excess}_P[i] = W_P[i] - W_{\text{FCFS}}[i]. \quad (2)$$

Writing $j \prec i$ when j is started before i , the work that overtook i and the work that i overtook are

$$\text{In}_i = \sum_{j \succ i, j \prec i} C_j, \quad \text{Out}_i = \sum_{j \prec i, i \prec j} C_j. \quad (3)$$

A policy P offers a *service guarantee* G if $\text{excess}_P[i] \leq G$ for every job i , on every arrival sequence, for all costs bounded by L and for every score, adversarial scores included. An operator states G in advance, and the guarantee is the proved statement that every job stays within it. This guarantee holds for each job on the realised sample path, not on average, and it assumes nothing about the quality of the score. Packet schedulers state additive per-flow bounds of this kind against a reference discipline [46]. Batch schedulers measure the same difference after the fact, against a fair start time [33]. However, the scheduling-with-predictions literature compares with an optimal policy [10,14]. We take FCFS as the reference because these services run it by default, so the excess is a quantity an operator can be held accountable for.

3.4. Metrics

The primary metric is the 99th percentile (p_{99}) of the waiting time of the jobs that arrive in the 24 hours before a deadline, written $p_{99_{dl}}$. This metric was fixed before any sealed part of a log was opened. With it we report the mean wait over all jobs and the largest excess, which is what a guarantee bounds. We also report the *harm*, the largest excess among the jobs that FCFS would have started within 1 s, that is, among jobs FCFS would not have delayed. A fourth quantity, the *firing rate*, is the share of dispatch decisions at which the guard of Section 5 overrides the base policy, weighted by queue length. Finally, we report the worst wait among heavy jobs, defined in Section 4.

Improvements are expressed as the fraction of the gap closed,

$$\text{gap closed}(P) = \frac{M_{\text{FCFS}} - M_P}{M_{\text{FCFS}} - M_{\text{SJF}}}, \quad (4)$$

where M_Q is $p_{99_{dl}}$ under policy Q and SJF orders the waiting jobs by their true costs. SJF cannot be deployed, since costs are unknown at arrival. On several non-preemptive servers SJF also minimises none of our metrics, so it is a reference and not an optimum. A gap closed of 1 means that P matches the p_{99} of SJF, and 0 that it matches FCFS. Gap closed is not a percentage reduction in waiting time, and the tables print the absolute waits beside it.

3.5. Steady-State Models and This Load

Stationary models give closed forms for the mean wait of each class of a non-preemptive priority queue [65, pp. 499–507]. The Schedule Ordered by Age-based Priority (SOAP) framework analyses every policy that ranks jobs by a function of their size class and age [66], a family that contains ranking by predicted cost. Both analyses need stationary arrivals, which deadline-driven load does not supply. On a synthetic Poisson stream the priority formula agrees with simulation to 0.4%, but on the same jobs at their recorded arrival times the measured mean wait is 10.7 to 13.7 times what the formula predicts (Supplementary Section S2.5). Thus, we use steady-state results as motivation only, and state every guarantee on the realised sample path.

4. Cost Prediction

The scheduler needs, for every arriving job, a number that says how long the job will run, computed from history alone. Users and tasks form a graph, because many users attempt the same task and each user attempts many tasks. A natural question is therefore whether a model that reads this graph predicts better than one that reads each job's own history. This section compares three kinds of model on the two grading logs of Section 7 and chooses the one the scheduler uses.

4.1. Targets and Measures

For ranking, the target is the conditional mean $\mathbb{E}[C_i \mid x_i]$. This mean minimises the expected mean wait of a batch on one server, by the pairwise-interchange argument of Section 5.1. The argument does not extend to the 99th percentile of the wait that we report, so whether this target also serves the percentile is tested in the queue (Section 8.3). The mix of jobs shifts between periods, so to compare predictors across periods we also score each model on a binary label. A job is *heavy* when its cost exceeds the 95th percentile of the training rows. On the main grading log this cut-off is 1.559 s, and heavy jobs are 1.31% of the test period against 5.00% of the training period. On the second log the cut-off is 1,314 ms and heavy jobs are 5.07%. For the heavy label we report the area under the receiver operating characteristic curve (AUROC). For the predicted cost we report the root mean

squared error (RMSE) of $\log(1 + C)$ and the Spearman rank correlation, since a ranking policy uses the order of the scores and not their level.

4.2. Protocol

Every model sees the same rows, the same split into training and test periods, and the same visibility rule (1). Historical features are computed by streaming the log in the order in which results became available and reading each user's and each task's state at the query instant. Recomputing these features from scratch gives 0 disagreeing rows in all eight configurations we tested. Each test period is predicted by a model fitted only on earlier periods and frozen before the period's first arrival, and the heavy cut-off is refitted on that model's own training rows. Delaying the results by up to ten minutes before they may enter the features leaves the ranking intact and worsens only the calibration. Supplementary Section S1 gives these audits.

The three kinds of model are listed in Table 1, which compares what each representation knows. Every model in that table is a classifier for the heavy label, trained on the terms 2018-1 to 2019-2, tested on 2022-2 and averaged over 3 seeds. The scheduler uses a different fit on M4's features, the expected-cost regressor of Section 5.1, refitted on rolling forward periods (Supplementary Section S3.9).

- Gradient-boosted trees [19] over five feature groups, namely the task's history (M1), the user's history (M2), static features of the submitted code (M3), all of these together (M4), and M4 with one-hop aggregates over related users and tasks (M5).
- A GRU over the user's earlier jobs, used alone (R1) and as a state vector added to M4's features inside the trees (R1S).
- A heterogeneous GNN of the R-GCN type [20], over users, tasks, the assignments that group tasks and the class cohorts that group users. It is tested as a ladder of five models. The first is a multilayer perceptron on M4's features (N0). The others are the graph architecture with every neighbour set empty and no group nodes (G0U), with the group nodes but no edges (G0), with the real edges (G1), and with the edges shuffled within each period (G2). G2 keeps each node's number of neighbours and changes only who they are, so if the edges carry information, G2 must lose it.

Table 1. Predictors on the main grading log (CodeBench), 40,844 test rows. AUROC is for the heavy label, and Spearman is on the predicted cost. The second grading log and the remaining measures are in Supplementary Table S2.

Kind	Model	Information	AUROC	Spearman
Trees	M1	task history	0.7960	0.219
	M2	user history	0.8591	0.302
	M3	static features	0.8632	0.290
	M4	all of the above	0.9283	0.312
	M5	M4 and one-hop aggregates	0.9302	0.344
GRU	R1	user's earlier jobs	0.9239	0.313
	R1S	M4 and the GRU state	0.9366	0.329
GNN	N0	M4, no graph	0.9209	0.278
	G0U	graph model, no edges or group nodes	0.9348	0.321
	G0	graph model, no edges	0.9325	0.324
	G1	graph model, real edges	0.9324	0.329
	G2	graph model, shuffled edges	0.9305	0.323

4.3. What the Comparison Shows

The graph does not help. One-hop aggregates barely change the trees' AUROC, since $M5 - M4$ is +0.0019 of AUROC on the main log and -0.0002 on the second. In the GNN

ladder, removing every edge (G0) or shuffling them (G2) leaves AUROC within 0.002 of G1. The paired intervals for $G1 - G0$, $-0.0001 [-0.0019, 0.0018]$, and for $G1 - G2$, $+0.0019 [-0.0000, 0.0042]$, both contain zero. The one step in the ladder that matters is N0 to G0U, $+0.0139 [+0.0090, +0.0195]$ of AUROC, and this step changes the encoder while the edges stay empty.

Tasks with little history of their own are where the neural models differ from the trees, and the edges do not cause the difference. On the 1,092 test rows whose task has fewer than five visible earlier results, G1 beats M4 by $+0.1464 [+0.0997, +0.1887]$ in Spearman correlation. The same comparison without any graph gives most of this: the perceptron N0, which sees only M4's features, beats M4 by $+0.1064$, the GRU R1 by $+0.1078$, and the edgeless G0U by $+0.1419 [+0.0994, +0.1803]$. Adding the real edges to G0U changes the correlation by $+0.0045 [-0.0163, +0.0248]$. G1 does beat its shuffled-edge control on these rows, by $+0.0282 [+0.0130, +0.0432]$, but because shuffled edges make the model worse than having none ($G2 - G0U$ is $-0.0238 [-0.0453, -0.0025]$), not because real edges make it better. On new tasks as over all rows, the relations between users and tasks add nothing measurable to the ranking that each entity's own features do not already give.

Prediction scores and queueing outcomes do not rank the models the same way. R1S has the highest AUROC, 0.9366. Its difference from M4's 0.9283 is $+0.0083 [+0.0038, +0.0131]$. Run through the queue with the same ranking objective as M4, however, R1S gains $+0.007 [0.003, 0.011]$ of the gap at utilisation 0.8, loses $-0.010 [-0.022, -0.002]$ at 0.5, and is unresolved at 1.0 (Section 7.2 defines the three loads). M5 has the higher AUROC of the two full tree models, and M5 loses in the queue at all three loads, by 0.012, 0.008 and 0.010 of the gap, each interval excluding zero. The scheduler therefore uses M4, chosen on the queueing outcome of validation periods rather than on prediction scores.

Which history matters varies by platform. The user's history leads on the main log, and the task's history leads on the second by 0.11 of AUROC (Supplementary Table S2). Supplementary Section S1.2 compares the feature groups on the four logs from outside education.

Prediction is cheap next to the queue. The median job on the main log runs for 204 ms, and charging each model's measured cost per job to the predictive policies changes the gap closed by less than 0.001 at every load (Supplementary Section S1).

5. Scheduling Algorithm

The scheduler has two layers. A base policy orders the queue by a score computed at arrival, and a guard limits how much work may overtake any waiting job. The guard accepts any base policy, and its guarantee does not depend on that policy behaving well.

5.1. Ranking by Expected Cost

When a server frees, the base policy starts the waiting job with the smallest score. For a batch on one server, the pairwise-interchange argument shows that ordering jobs by their conditional mean costs minimises the expected mean wait. However, this conclusion does not carry over to $k \geq 2$ servers. On two servers, let job u cost 2 and job w cost 3, and let job v cost 0.1 with probability 0.9 and 100 with probability 0.1. Ordering by conditional mean starts u and w and gives an expected mean wait of $2/3$, whereas starting u and v gives 0.096667. We therefore treat expected-cost ordering as an empirically motivated base policy, not as a claim of optimality.

Squared error on raw seconds and the Tweedie deviance both have the conditional mean as their population optimum. Their fitted orders can nevertheless differ with finite model capacity, link function and regularisation, so we compare the objectives empirically,

with the features and model of Section 4 held fixed. We fit the expected-cost policy, shortest predicted job first on the expected-cost score (SPJF-E), on raw seconds with a Tweedie loss of power 1.5, which accommodates mass near zero and a long right tail [67,68]. For the log-scale policy, shortest predicted job first on the log-scale score (SPJF-log), we fit the same pipeline by squared error on $\log(1 + C)$. By Jensen's inequality, the back-transform of this fit is no larger than the conditional mean.

We use only the order of a score. Its level does not transfer between periods, because the share of heavy jobs differs between training and test periods by a factor of several, so the scheduler never reads a score as a time or a threshold. Section 8.3 compares these and other objectives in the queue. No choice of score bounds the worst case. Whatever the score, a job it ranks late, because the job is long or because its cost is overestimated, can be passed again and again by later arrivals ranked ahead of it, and only the guard below bounds that.

5.2. The Overtake-Budget Guard

Fix a base policy A . For every waiting job q and every time t let

$$\text{over}[q](t) = \sum \{ C_j : j > q, j \text{ has completed by } t \}, \quad (5)$$

the total cost of the jobs that arrived after q and have finished by t . Such a job ran while q waited, so it overtook q . Each waiting job has a budget $\text{budget}(q, t)$ on this quantity, and the job is *eligible* when its account has reached its budget, $\text{over}[q](t) \geq \text{budget}(q, t)$. Whenever a server is free and the queue is not empty, the guard starts the earliest-arrived eligible job if there is one, and otherwise lets A choose (Algorithm 1 and Figure 2). Serving the earliest-arrived eligible job matters when budgets differ between jobs. It ensures that whenever a job j starts ahead of an earlier-arrived waiting job i , the job i was not eligible at that instant, and the per-job bound of Section 6 rests on this step. With a constant budget B the account is non-increasing in arrival order, so the rule reduces to a test on the head of the queue, and $B = 0$ gives FCFS.

The guarantee reads the budget only through an upper bound on it. If $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job at every instant, then every job waits at most $W_{\text{FCFS}} + B_{\max}/k + (3 - 2/k)L$, whatever the base policy, the arrivals and the predictions (Theorem 7). Below the cap, the budget decides how much of the base policy's benefit survives and which jobs pay for it. We study the family

$$\text{budget}(q, t) = \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max}), \quad (6)$$

with $B_0, \gamma \geq 0$, $0 \leq \eta < 1$ and n_q the number of jobs waiting when q arrives. We study three shapes in this family, namely *constant* ($\gamma = \eta = 0$), *age-relative* ($\eta > 0$) and *queue-length* ($\gamma > 0$). In the age-relative shape, η is the share of the system's capacity, over a job's own wait, that may go to jobs that arrived after it, so the allowance grows with the wait the job has already had. The term γn_q widens the allowance only for jobs that arrive into a long queue. All of B_0 , γ and $B_{\max}$ are measured in work and scale with k .

5.3. Setting the Guard

An operator states a bound G , the number of seconds by which any job may exceed its FCFS wait, and sets $B_{\max} = k(G - (3 - 2/k)L)$ (Corollary 8). No data are needed for this step. A bound of at most $(3 - 2/k)L$ can be guaranteed this way only with $B_{\max} = 0$, which is FCFS. The shape parameters B_0 , γ and η do not affect the bound, and we choose them on validation data by a rule fixed in advance (Section 7.3). At $G = 600$ s this rule

Algorithm 1 Overtake-budget guard wrapped around a base policy A

```

1: parameters: cap  $B_{\max} \geq 0$ ; shape  $B_0, \gamma \geq 0, \eta \in [0, 1)$ ; base policy  $A$ 
2: state: waiting set  $Q \leftarrow \emptyset$ ;  $\text{over}[q] \leftarrow 0$  for every job  $q$ 
3: while some job has not started do
4:    $t \leftarrow$  next event time
5:   for all jobs  $c$  completing at  $t$  do  $\triangleright C_c$  becomes known here and nowhere earlier
6:     for all  $q \in Q$  that arrived before  $c$  do
7:        $\text{over}[q] \leftarrow \text{over}[q] + C_c$ 
8:     end for
9:   end for
10:  for all jobs  $i$  arriving at  $t$  do
11:     $n_i \leftarrow |Q|$ ;  $Q \leftarrow Q \cup \{i\}$ ;  $\text{over}[i] \leftarrow 0$ 
12:  end for
13:  while a server is free and  $Q \neq \emptyset$  do  $\triangleright$  dispatch decision
14:     $E \leftarrow \{q \in Q : \text{over}[q] \geq \min(B_0 + \gamma n_q + \eta k(t - a_q), B_{\max})\}$ 
15:    if  $E \neq \emptyset$  then
16:       $j \leftarrow$  the earliest-arrived member of  $E$   $\triangleright$  guard fires
17:    else
18:       $j \leftarrow A(Q, t)$   $\triangleright$  base policy decides
19:    end if
20:    start  $j$  on the free server;  $W[j] \leftarrow t - a_j$ ;  $Q \leftarrow Q \setminus \{j\}$ 
21:  end while
22: end while

```

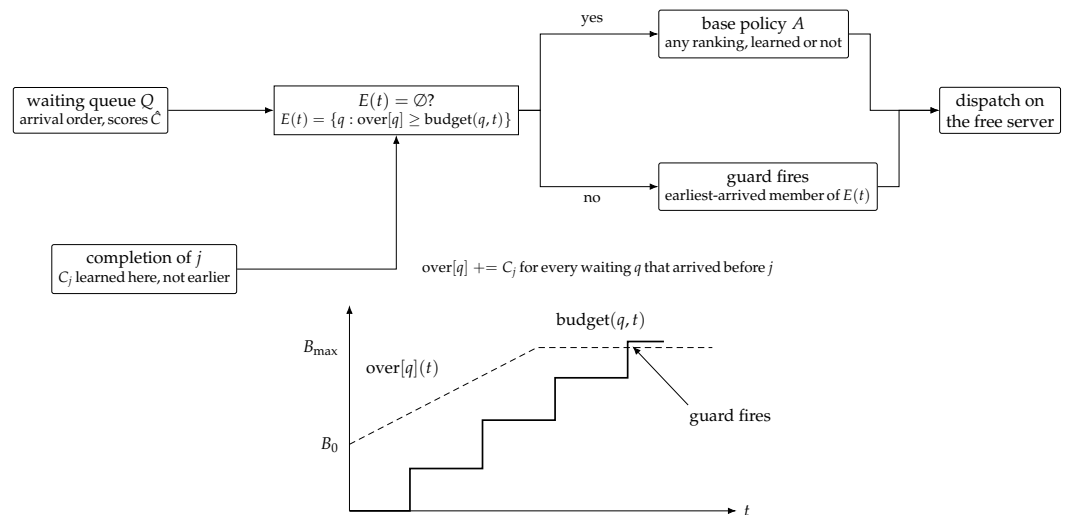


Figure 2. The guard of Algorithm 1 wrapped around a base policy (above), and the account it keeps for one waiting job q (below). Only completed work is charged, because a job's cost is unknown until it finishes, so $\text{over}[q]$ rises by one step at each completion of a job that arrived after q . When the account reaches the budget, q becomes eligible, and the guard starts the earliest-arrived eligible job, which need not be q . The budget drawn is the age-relative shape, rising from B_0 at rate ηk and capped at $B_{\max}$; the bound depends on the cap alone, so any budget below the dashed ceiling carries the same guarantee. The lower panel is a schematic.

selects the age-relative shape with $B_0 = 30k$ s of work and $\eta = 0.75$ (Supplementary Table S5).

The guard charges the executed cost of an overtaker when it completes, because a job's cost is known only then. A job still running contributes nothing yet. When the last overtaker of a job i starts, up to $k - 1$ earlier overtakers may still be running, and the one starting is not charged either. The work that actually passes i can therefore exceed the budget by at most kL . This is what the additive term of the guarantee pays for, and it is why the guard needs no prediction to enforce its bound. With a Fenwick tree over completed work and a segment tree over the waiting jobs, each event costs $O(\log n)$, and all budget arithmetic is done in integer microseconds, so the eligibility test is exact (Supplementary Section S5).

5.4. Compared Policies

Table 2 names the policies compared in Section 8. Our method is Guard(G), the guard with bound G wrapped around SPJF-E. The other rows separate its parts or test it.

Table 2. Policies compared in the experiments.

Name	Role
FCFS	Deployed baseline and the reference of the guarantee
SJF	Orders by true cost, cannot be deployed, and is the reference of the gap closed
SPJF-E	Orders by the expected-cost score of Section 5.1, unguarded
SPJF-log	Orders by the log-scale score, unguarded
Guard(G)	SPJF-E inside the guard with bound G and the selected shape (the method)
Guard-fixed(G), age(G), -queue(G)	The same guard with the best constant, age-relative or queue-length shape
Fixed(G)	The same guard with the whole cap as budget, budget $\equiv B_{\max}$
Skip(G)	A guard that counts overtakes instead of work, charged at start, at the count that gives bound G
Timeout(θ)	A maximum waiting time, which serves the earliest-arrived job once it has waited θ (Section 6.6)
Corrupted scores	SPJF-E and Guard(G) under a reversed, a random and an inverted-tail score

Every simulation that includes a guard checks the per-job bound of Section 6 on every job while it runs (Section 8.5).

6. Theory

Everything in this section is stated for one fixed run: one arrival sequence, one vector of costs and one realisation of whatever randomness the base policy uses. No stationarity, no independence and no assumption about the scores enters a proof, and the upper bounds hold even against a base policy that knows the future. An identity ties a job's delay relative to FCFS to the work that overtook it (Sections 6.2–6.4), two bounds turn it into the guarantee of Algorithm 1 (Section 6.5), and three further results cover a maximum waiting time, an absolute bound and completion reports that arrive late (Sections 6.6–6.8). Full proofs and the extremal constructions are in Supplementary Sections S4 and S8.

6.1. Conventions

We use the model of Section 3 and fix three readings the proofs depend on. The jobs started at one instant form a *phase*, ordered as the policy starts them, and $j \prec i$ refers to the concatenation of the phases. Within a phase, FCFS starts the jobs in arrival order. A later-arriving job started in the same phase as i but ahead of it counts as overtaking i and

contributes its whole C_j to In_i , although it executes nothing while i waits. Remaining work at a_i is read after the arrivals at a_i .

6.2. The Workload Gap between Two Policies

Write $U_Q(t)$ for the unfinished work at time t under policy Q , counting every job with $a_j \leq t$.

Lemma 1 (Workload comparison). *Let Q_1 and Q_2 be non-preemptive work-conserving k -server policies run on the same input. Then $|U_{Q_1}(t) - U_{Q_2}(t)| \leq (k-1)L$ for every t . For $k \geq 2$ the inequality is strict, $|U_{Q_1}(t) - U_{Q_2}(t)| < (k-1)L$, and the constant is the exact supremum, approached but never attained. At $k = 1$ both sides vanish, because one server executes work at rate 1 whenever at least one job is present, so every work-conserving policy has the same unfinished work at every instant.*

Proof sketch of Lemma 1. The difference $D = U_{Q_1} - U_{Q_2}$ is continuous and piecewise linear. Whenever U_{Q_1} exceeds $(k-1)L$, more than $k-1$ jobs are present under Q_1 , every server of Q_1 is busy, and D cannot increase. Tracking D back to the last instant at which U_{Q_1} was at most $(k-1)L$ gives the bound. Supplementary Sections S4.1 and S8.1 give the proof, the strictness and the construction that shows the constant cannot be lowered. $\square$

The argument that a work-conserving k -server system can fall behind only by what $k-1$ servers hold is the one used to compare a multiserver system with a single fast server in the analysis of shortest remaining processing time (SRPT) [69,70], where it bounds the work below a size cutoff. Here we use it to compare any two non-preemptive work-conserving policies over all of the work. FCFS also minimises the unfinished-work vector in a stronger order, but only among policies whose assignment of jobs to servers ignores their costs [71], which the policies compared here do not.

6.3. The Net-Overtake Identity

For a job i and a policy Q , let R_i^Q be the remaining work at time a_i , under Q , of the jobs that arrived before i , and let β_i^Q be the remaining work at s_i^Q of the jobs started before i that are still running then. Write

$$\Delta_i = k(W_P[i] - W_{\text{FCFS}}[i]) - (\text{In}_i - \text{Out}_i). \quad (7)$$

Theorem 2 (Net-overtake identity). *For every non-preemptive work-conserving k -server policy P , every input and every job i ,*

$$\Delta_i = (R_i^P - R_i^{\text{FCFS}}) - \beta_i^P + \beta_i^{\text{FCFS}}, \quad |\Delta_i| \leq 2(k-1)L, \quad (8)$$

with strict inequality for $k \geq 2$. Equivalently, $|W_P[i] - W_{\text{FCFS}}[i] - (\text{In}_i - \text{Out}_i)/k| \leq (2 - 2/k)L$. At $k = 1$ the error term vanishes and $W_P[i] - W_{\text{FCFS}}[i] = \text{In}_i - \text{Out}_i$ exactly.

Proof sketch of Theorem 2. Job i waits throughout $[a_i, s_i^P]$, so every server is busy there and exactly $k W_P[i]$ units of work are executed, none of it on i . Splitting that work by arrival order gives

$$k W_P[i] = R_i^P + \text{In}_i - \text{Out}_i - \beta_i^P. \quad (9)$$

The jobs counted in β_i^P run on distinct servers other than the one that takes i , so $0 \leq \beta_i^P \leq (k-1)L$. Under FCFS, $\text{In}_i = \text{Out}_i = 0$, and Lemma 1 applied at a_i bounds $|R_i^P - R_i^{\text{FCFS}}|$ by $(k-1)L$. Subtracting the two instances of (9) gives the identity. Each direction uses only two of the three ranges, which is why the slack is $2(k-1)L$ and not $3(k-1)L$. Supplementary Section S4.3 gives the proof in full. $\square$

Corollary 3 (A work budget is sufficient). *If a policy keeps $\text{In}_i - \text{Out}_i \leq Z$ for every job, in particular if it keeps $\text{In}_i \leq Z$, then $\text{excess}_P[i] \leq Z/k + (2 - 2/k)L$ for every job.*

Proof of Corollary 3. By Theorem 2, $k \text{excess}_P[i] \leq \text{In}_i - \text{Out}_i + 2(k-1)L \leq Z + 2(k-1)L$. Dividing by k gives the claim. $\square$

So a scheduler that never lets more than Z units of work pass a waiting job holds every wait within $Z/k + (2 - 2/k)L$ of its FCFS wait, whatever it ranks by. The constant cannot be improved.

Theorem 4 (Tightness of Theorem 2). *For every fixed $k \geq 2$, service bound $L > 0$ and $\varepsilon > 0$ there are inputs, work-conserving policies P and jobs i with $\Delta_i > 2(k-1)L - \varepsilon$, and there are others with $\Delta_i < -2(k-1)L + \varepsilon$. Hence $\sup |\Delta_i| = 2(k-1)L$, a supremum that is never attained.*

Proof sketch of Theorem 4. Both witnesses start from a geometric cascade that drives $U_P - U_{\text{FCFS}}$ to $(k-1)L(1 - ((k-1)/k)^m)$ in m rounds and then place a victim so that FCFS starts it at once while P holds it back (upward), or the mirror image (downward). The instances need hundreds of jobs and are rescaled to keep L fixed. Supplementary Section S8.2 gives both in full. Non-attainment follows from Lemma 1. $\square$

In the downward direction, half of the extremal value comes from the convention of Section 6.1 that charges a same-phase overtaker in full. Supplementary Section S4.3 gives the variant that charges executed work only.

6.4. A Work Budget Is Also Necessary

A policy that guarantees anything about individual jobs is forced to bound the work that overtakes them.

Lemma 5 (Outbound work under a guarantee). *Let i be a job, and suppose $\text{excess}_P[i] \leq G$ and $\text{excess}_P[j] \leq G$ for every job j counted in Out_i . Then $\text{Out}_i \leq k(G - \text{excess}_P[i] + L)$.*

Proof sketch of Lemma 5. Every job counted in Out_i starts inside a window of length $G - \text{excess}_P[i]$, and of the jobs one server starts inside such a window all but the last complete inside it (Supplementary Section S4.7). $\square$

Corollary 6 (A work budget is necessary). *If P satisfies $\text{excess}_P[j] \leq G$ for every job on some input, then $\text{In}_i \leq kG + (3k-2)L$ for every job i .*

Proof of Corollary 6. Theorem 2 gives $\text{In}_i \leq k \text{excess}_P[i] + \text{Out}_i + 2(k-1)L$. Lemma 5 bounds Out_i , and the terms in $\text{excess}_P[i]$ cancel. $\square$

The two corollaries say that bounding In_i and offering a per-job guarantee are the same requirement, up to an additive $O(kL)$. A bound on the number of overtakes, as in the mechanisms of Section 2, is not necessary for a guarantee, and it gives one only by charging each pass the worst case L . That charge makes it looser than a work budget by the ratio of L to the mean cost, 150 on the main grading log ($L = 60$ s against a mean cost of 0.40 s). Supplementary Section S4.4 proves both halves and gives the conversion from a count to a bound.

6.5. The Guarantee of the Guard

Nothing below restricts the base policy, which may be randomised, learned or clairvoyant, and nothing restricts the budget rule beyond the cap: it may depend on the job, decrease over time, and take different values at two decisions made at the same instant.

Theorem 7 (Guard bound). *Fix a base policy A , a cap $B_{\max} \geq 0$, and a budget rule that respects the cap, that is, $0 \leq \text{budget}(q, t) \leq B_{\max}$ for every waiting job q and every dispatch decision at time t , and run Algorithm 1 with that rule. For every input and every job i , writing $W = W_P[i]$ for the wait under the guard,*

$$In_i < B_{\max} + kL, \quad (10)$$

$$W < W_{\text{FCFS}}[i] + B_{\max}/k + (3 - 2/k)L, \quad (11)$$

where (10) is asserted whenever i has at least one overtaker.

Proof sketch of Theorem 7. When the last overtaker of i starts, i is waiting and not eligible, so $\text{over}[i]$ is below its budget there. The overtakers started earlier and not yet finished occupy at most $k - 1$ servers, and the last one adds a further L , which gives (10). Theorem 2 with $\text{Out}_i \geq 0$ turns it into (11). Supplementary Section S4.5 gives the proof in full. $\square$

Corollary 8 (The cap for a given bound). *Setting $B_{\max} = k(G - (3 - 2/k)L)$ makes Algorithm 1 offer the service guarantee G , whatever the budget shape below the cap and whatever the base policy A . The choice is available when $G \geq (3 - 2/k)L$, and gives a positive budget only when the inequality is strict. At equality $B_{\max} = 0$ and the guard is FCFS.*

Proof of Corollary 8. Substitute $B_{\max}$ into (11). $\square$

The per-job promise also bounds the largest flow time, the quantity most scheduling results for this setting bound. Write $F_i = W[i] + C_i$ for the flow time of job i and F^* for the smallest largest flow time $\max_i F_i$ that any schedule of the input on k servers achieves, with or without interruption.

Corollary 9 (Largest flow time). *Under the cap of Corollary 8, every job has $W_P[i] + C_i < (3 - 2/k)F^* + G$.*

Proof of Corollary 9. We show $W_{\text{FCFS}}[i] + C_i \leq (3 - 2/k)F^*$; adding the guarantee G gives the claim. Let S_i be i 's start under FCFS and t' the last instant at or before a_i at which a server is idle. No job waits at t' , and all k servers are busy on (t', S_i) , since before a_i none is idle after t' and from a_i on i waits. That busy time is filled by the at most $k - 1$ jobs running at t' , each with remaining cost at most $p = \max_j C_j \leq F^*$, and by jobs that arrived in $[t', a_i]$ and started before i , of total cost W_A . Hence $k(S_i - t') \leq W_A + (k - 1)p$. Every schedule must run these jobs and i after t' and finish them by $a_i + F^*$, so $W_A + C_i \leq k(a_i + F^* - t')$. Together, $S_i - a_i \leq F^* - C_i/k + (1 - 1/k)F^*$, and $F_i \leq (2 - 1/k)F^* + (1 - 1/k)C_i \leq (3 - 2/k)F^*$. $\square$

At $k = 1$ the factor is 1, so FCFS minimises the largest flow time on one server and the guard stays within G of that optimum.

The proof reads the budget only through the cap, so the shape below the cap is free. The age-relative shape with one constant B_0 for every job also gives a multiplicative bound, $(1 - \eta)W < W_{\text{FCFS}}[i] + B_0/k + (3 - 2/k)L$. The queue-length shape does not, by a margin that grows with γ (Supplementary Sections S4.5 and S4.6). Neither constant of Theo-

rem 7 can be improved: with a constant budget the term $B_{\max}/k$ is reached exactly, and $(3 - 2/k)L$ is the exact supremum at every k , approached by a cascade (Supplementary Section S8.3). If the service enforces a run-time cap known at arrival, charging it at the start and refunding the difference at completion removes L from the constant (Supplementary Section S7.6). Finally, a guard that never fires leaves the base policy's run unchanged, which can be checked on that run before the guard is deployed (Supplementary Section S4.2).

6.6. A Maximum Waiting Time Gives the Same Guarantee

Deployed schedulers guard against starvation with a maximum waiting time, and recent inference servers promote a request once its wait passes a threshold [56,57], with no stated bound. In the simplest form, once the earliest-arrived waiting job has waited θ it is started next, and until then the base policy decides. We call this rule Timeout(θ). It reads arrival times and a clock, never a cost.

Proposition 10 (Timeout bound). *Fix a base policy and $\theta \geq 0$. For every input and every job i with at least one overtaker, Timeout(θ) gives $\text{In}_i < k(\theta + L)$, and every job waits*

$$W < W_{\text{FCFS}}[i] + \theta + (3 - 2/k)L. \quad (12)$$

Proof sketch of Proposition 10. From $a_i + \theta$ on, the earliest-arrived waiting job has waited at least θ , so every job started before s_i arrived before i . Each overtaker of i therefore starts before $a_i + \theta$ and runs at most L , and on each server the overtakers fit in $[a_i, a_i + \theta + L)$. Theorem 2 with $\text{Out}_i \geq 0$ gives the bound (Supplementary Section S4.11). $\square$

At $\theta = G - (3 - 2/k)L$ the clock gives the bound that the guard gives at $B_{\max} = k\theta$. Unlike Theorem 7, the constant of Proposition 10 is not shown to be tight: the worst ratio of excess to bound found by search is 0.996 at $k = 1$ and 0.756 to 0.829 for $k = 2$ to 4 (Supplementary Section S4.11), so Section 8.4 also compares the two rules at equal realised worst excess. The rules differ in what they charge a waiting job for. The clock charges it for all the work the servers do while it waits, the guard only for completed work of jobs that arrived after it. Near saturation, a job that arrives behind a backlog is charged for the backlog, the clock fires for most of the queue, and the schedule falls back to arrival order at the loads where ordering by cost pays most.

The two rules can be combined: start the earliest-arrived job if it has waited θ , and otherwise run Algorithm 1. Both bounds then hold at once. The order matters, since serving the guard's eligible job first can pass over a job that has already waited θ (Supplementary Section S4.11). The clock's half reads no completion report, so the combined rule keeps (12) when reports arrive late or never.

6.7. An Absolute Bound from the Arrival Log

A bound relative to FCFS is a margin over a schedule that is not run. The arrival log alone gives an absolute bound, which an operator can state as a number of seconds. Let V_i^- be the work that a single server of rate k , fed the same jobs, would still hold at a_i from the jobs that arrived before i : $V_1^- = 0$ and $V_i^- = \max(0, V_{i-1}^- + C_{i-1} - k(a_i - a_{i-1}))$. The largest value $\sigma = \max_i (V_i^- + C_i)$ is the depth of the smallest token bucket of rate k that the arrivals, counted in work, conform to, which is the burst parameter of network calculus [72,73].

Proposition 11 (Absolute bound). *For every non-preemptive work-conserving policy and every job i , $k W[i] \leq V_i^- + (k - 1)L + \text{In}_i$. Hence $W_{\text{FCFS}}[i] \leq (V_i^- + (k - 1)L)/k$, and under Algorithm 1 with cap $B_{\max}$*

$$W < \frac{V_i^- + B_{\max}}{k} + \left(2 - \frac{1}{k}\right)L \leq \frac{\sigma + B_{\max}}{k} + \left(2 - \frac{1}{k}\right)L. \quad (13)$$

Proof sketch of Proposition 11. Equation (9) gives $k W[i] \leq R_i + \text{In}_i$. A work-conserving k -server queue never holds more than $(k - 1)L$ above the single fast server: whenever it does, all k servers are busy and it drains at the same rate. Hence $R_i \leq V_i^- + (k - 1)L$, and Theorem 7 bounds In_i (Supplementary Section S4.11). $\square$

At $k = 1$ the fast server is the queue itself, $V_i^- = W_{\text{FCFS}}[i]$, and the proposition is Theorem 7 again. An operator who guarantees that no job waits longer than D , over a period whose arrivals fit a bucket of depth σ , sets $B_{\max} = kD - \sigma - (2k - 1)L$: the capacity over the deadline, less the burst, less $2k - 1$ time limits. The cap is non-negative only when $D \geq \sigma/k + (2 - 1/k)L$, and a shorter deadline cannot be guaranteed this way. The clock obeys the same bound with $k\theta$ in place of $B_{\max}$. Supplementary Section S4.11 relates the statement to earlier sample-path comparisons with a fast server, some of which were shown to be false [74].

6.8. Completion Reports That Arrive Late

Theorem 7 assumes that a completion is charged before the next dispatch decision. In a distributed service the servers are separate machines that report their completions to the scheduler over a network, and a server may take its next job before its report has been counted. Suppose every report reaches the scheduler's counter within δ of the completion, while servers still take a job the instant they free.

Proposition 12 (Late charges). *Under Algorithm 1 with a shared counter and charges delayed by at most δ , every job i with an overtaker has $\text{In}_i < B_{\max} + kL + k(L + \delta)$, and*

$$W < W_{\text{FCFS}}[i] + B_{\max}/k + (4 - 2/k)L + \delta. \quad (14)$$

For $\delta > 0$ the bound is attained in the limit at $k = 1$ and at $k = 2$. If each server applies its own completion report in the same step as its next pull, and only other servers' reports travel late, Theorem 7 holds unchanged for every δ , lost reports included.

Proof sketch of Proposition 12. When the last overtaker starts, the charged part is below the budget as before. The uncharged overtakers are those still running, as in Theorem 7, and those that finished within the last δ and whose report has not arrived. On one server the latter are at most one job that started before that window and jobs that fit inside it, so they hold less than $L + \delta$. When a server reports before it takes its next job, an uncounted completion can belong only to a server that finished at the decision instant itself and has not yet taken a job, so it holds nothing in service. Supplementary Section S4.12 gives both proofs and the instances that attain the bound. $\square$

At $k = 1$ any $\delta > 0$ already costs a whole L , because the single server takes its next job the instant it finishes one, before that job's report can arrive. The remedy is a protocol change rather than a smaller budget: a server that counts its own completion before taking the next job restores Theorem 7, and the combined rule of Section 6.6 keeps its clock bound for any delay. The model counts a dispatch as instantaneous. A server that must wait a network round trip for its next job is idle while jobs wait, and the bound of a job then grows by the server-idle time accumulated while it waits, divided by k (Section 6.9). If

each server keeps its own view of the counter, in which it sees its own completions at once and other servers' within δ , the excess is below $B_{\max}/k + (3 - 2/k)L + (1 - 1/k)(L + \delta)$, and at $k = 2$ this bound is also attained in the limit (Supplementary Section S4.12).

The guard needs no coordination beyond the reports. At any instant over $[q]$ is the total cost of the reported completions of jobs that arrived after q , a function of the set of reports received and of the arrival order, and not of the order in which the reports came. Two replicas of the scheduler that share the arrival order and have received the same reports therefore hold the same counters and the same set of eligible jobs, so they agree whenever the guard overrides the base policy; they agree on every decision when the base policy is deterministic given the same history. The bound needs only the first agreement: a replica that lags the reports by at most δ is a scheduler whose charges arrive up to δ late, the case of Proposition 12. A report counted twice can only raise $\text{over}[q]$. The proof of Theorem 7 uses only that the counted total of i was below its budget when i 's last overtaker started, so the true total was below it too, and the bound holds with duplicated reports.

6.9. Which Assumptions Can Be Dropped

We may replace the global bound L by the largest cost seen so far, read at the right instant. Servers may run at different speeds, with the budget divided by total capacity, and a fixed setup paid before every job is absorbed into its cost. Work conservation cannot be dropped. If a server may idle while jobs wait, the identity fails by exactly the forced idle time. Supplementary Section S4.10 states these relaxations with their sharp forms.

6.10. A Floor of Order L , and What a Guarantee Costs

No budget removes the last L of the additive constant, because a job can be displaced before anything has completed, and at that instant a guard that charges completed work has nothing to charge.

Proposition 13 (Floor). *Fix a budget rule that assigns a positive budget to every waiting job at the first dispatch decision. Then there are a base policy A that observes a job's cost only at its completion, and an input, such that Algorithm 1 run around A with that rule leaves some job with excess $\geq L$. Consequently no bound $G < L$ can be guaranteed by a wrapper of this family that holds for every base policy, or for one prediction-ranked base policy under arbitrary predictions, which is the generality in which Theorem 7 is stated.*

Proof sketch of Proposition 13. Put $k + 1$ jobs of cost L into an empty system at $t = 0$. Nothing has completed, so no job is eligible, the guard accepts the base policy's choices, and a base policy that starts the last-arrived job in the first phase pushes back a job that FCFS starts at once, by L . Supplementary Section S4.8 gives the proof and shows that the hypothesis can be neither dropped nor weakened. $\square$

At one server the floor is the whole story: with a constant budget $B > 0$ the bound $B + L$ of Theorem 7 is the exact supremum of the excess (Supplementary Section S8.4). The guarantee also caps how much reordering can gain at all.

Theorem 14 (Price of the guarantee). *Take m jobs of cost L and n jobs of cost $c < L$, the long ones arriving first, all at $t = 0$, and let P be a non-preemptive work-conserving policy with $\text{excess}_P[j] \leq G$ for every job. Define $D_0 = mn(L - c)/k$, $H = 2(1 - 1/k)L(m + n)$ and $\alpha = (kG + (3k - 2)L)/(nc)$. If $D_0 > H$, equivalently $mn(L - c) > 2(k - 1)L(m + n)$, then*

$$\frac{\overline{W}_{\text{FCFS}} - \overline{W}_P}{\overline{W}_{\text{FCFS}} - \overline{W}_{\text{SJF}}} \leq \alpha + \varepsilon_+, \quad \varepsilon_+ = \frac{H(1 + \alpha)}{D_0 - H}. \quad (15)$$

For fixed k, L, c and G , the finite-sample term is $O((m+n)/(mn))$.

Proof sketch of Theorem 14. Count the inverted pairs of long and short jobs in two ways, once through the mean waits, which reads the interchange argument behind Smith’s rule [75] on this input, and once through Corollary 6 applied to each long job. Supplementary Section S8.5 gives the proof. $\square$

The bound is capacity times G , divided by the work that wants to overtake. It is linear in G and reaches 1 when kG covers the total cost of the short jobs, and because it applies to every policy with the guarantee, it describes the trade-off an operator faces rather than a weakness of this guard. At $k = 1$ the achievable fraction is known exactly, $\lfloor G/c \rfloor / n$ capped at 1, and the guard around exact SJF meets the bound of Theorem 14 on this family up to $2(3k-2)L/(nc)$ and finite-sample terms (Supplementary Sections S8.5 and S8.6, with an exhaustive check over 1,161 parameter settings).

7. Workloads and Experimental Setup

7.1. The Workloads

We use six logs of real jobs and one log of student activity (Table 3). Our main log comes from CodeBench, the programming-course platform of the Universidade Federal do Amazonas in Brazil, which records every run of a student’s code together with its duration [21]. A second grading log comes from ACcoding, an online judge used in university programming courses [22]. ACcoding’s public release has no timestamps, only submission identifiers. Without timestamps, we keep the real order and costs of its jobs and draw their arrival times, so its results test the ordering mechanism but not waiting in seconds.

Four logs come from outside education. The Azure Functions trace of 2021 covers two weeks of calls to functions that run on demand [5,6]. Two pools of test machines that Mozilla uses for the Firefox browser, which we call CI pools A and B, recorded the waits we use to check our simulator [23,24]. In Intel’s Netbatch compute farm [25,26], single jobs run for days, and this farm is the case where our method does not apply.

Finally, the Open University Learning Analytics Dataset (OULAD) records the daily activity of students in online courses [76]. Because it contains no jobs, we use it only to check that activity crowds before deadlines outside our own grading logs.

Table 3. The logs, what each records and what we use it for. “Arrivals” says whether arrival times are recorded, reconstructed from the recorded end time and duration, or drawn; “Waits” whether each job’s queuing wait is recorded.

Log	Use	Size	Arrivals	Waits
CodeBench [21]	main scheduling log	4,611,325 runs; 863,149 graded submissions in the development period	reconstructed	no
ACcoding [22]	ordering on a second grading platform	4,046,652 submissions, 3,237,322 outside the sealed block	drawn	no
Azure Functions 2021 [5]	a cloud workload	1,980,951 calls over 14 d	reconstructed	no
CI pool A [23,24]	simulator check; light tail	191,970 runs over 21 d	recorded	yes
CI pool B	second simulator check	30,702 runs over 7.0 d	recorded	yes
Intel Netbatch [25,26]	a case where the method does not apply	9,054,066 jobs over 30 d, 8,441,207 serial with positive run time	recorded	yes
OULAD [76]	deadline crowding elsewhere	9 course presentations, daily counts	—	—

Three features of the main log shape how we use it, and Supplementary Section S2 gives the full audit. Since the timestamp of a run is the moment its result was written, we reconstruct the arrival as $a_i = t_i - C_i$ and treat the recorded stamp as done_i . This is the time

the run started. CodeBench runs each student's code in its own container (Section 7.2), so we take it as the submission time, and the log does not record the delay between the two. Supplementary Section S3.15 reads the stamp the other way, as the time of receipt, for an earlier configuration of the method.

Stamps are whole seconds while durations are in microseconds, which lets a model read the fractional part of the gap to the previous record and learn from it something it should not know. Given only that part, a classifier separates heavy from light jobs with AUROC 0.8091. When each stamp is jittered deterministically this AUROC falls to 0.4499, so the fractional part no longer ranks heavy jobs above light ones, and the forward models move by less than 0.008.

No run after 2019-08-28 lasts longer than 60 s, although the platform documents no timeout. Hence we take the periods after that date, cap each duration at the observed ceiling, $C^{\text{cap}} = \min(C, 60)$, and impose $L = 60$ s in our pool, as a shared service would (Supplementary Table S3). The logs from outside education document no time limit. For them we take L to be the 99.9th percentile of running times in the log's training period and cap longer jobs at it, an analysis choice that a deployment would have to enforce by stopping those jobs (Supplementary Table S3).

7.2. A Shared Pool Built from the Log

CodeBench runs each student's code in a separate container [77], so its log records how long each run took but not a shared queue. Our experiments therefore replace the containers with a pool of k servers fed by the recorded runs, a queue of our own construction.

Three things support this construction. A fixed pool of judging machines on one queue is a standard deployment [7,8]. On the same demand, one container per student keeps 1,636 containers alive at the p99, and each of them runs code for 0.12% of its lifetime, whereas a pool of 14 servers holds the p99 of waits before deadlines under 1 s. Finally, the simulator reproduces waits that someone else recorded. When fitted on one part of each CI log, it returns the other part's mean wait, hourly profile and per-job order (Supplementary Section S2.3). None of this shows that the original platform ever queued, and we make no claim about waiting there.

A single course in a single term does not load even one server, so we overlay several. From the 40 class-terms of the development period, each holding the submissions of one class in one academic term, we superimpose 44 copies. The copies are aligned by weekday and hour, and each is shifted by a whole number of weeks drawn at random. Overlaying whole terms keeps the daily and weekly cycles and each job's position relative to its own deadline, which compressing the gaps between arrivals would not. In all, 17,634,760 jobs arrive over 30 weeks, and 4,219,556 of them arrive in the 24 hours before an assessment deadline.

Three loads share this trace and differ only in the number of servers. Four of the five overlays, which differ only in the drawn week shifts, use $k = 8, 5$ and 4 . On the fifth, whose busy hours carry the least work, k is $7, 5$ and 4 , so tables that average over overlays print $k = 7.8$ at the lightest load. Averaged over the five overlays, the utilisation of the busiest hours is $0.502, 0.783$ and 0.979 , and we call the three loads $\rho = 0.5, 0.8$ and 1.0 . These loads are $0.529, 0.847$ and 1.059 on the first overlay alone.

The five overlays are one construction and not five platforms. For intervals we use 95% percentile intervals from 2,000 resamples of whole weeks, drawn once and shared by every policy so that differences between policies are paired. Such intervals cover variation over time, not the choice of terms or courses. Resampling whole class-terms instead and rebuilding the overlays each time widens them several-fold. At the busiest load Guard(600)

then closes 0.801 of the gap with interval $[0.328, 0.913]$, and its difference to unguarded ordering still excludes zero (Supplementary Table S18).

Arrivals follow the course calendar. When the number of runs per class is predicted a week ahead, the log-scale RMSE is 1.243 from recent counts alone, 1.159 from calendar features alone and 1.112 from both. Adding the calendar thus changes the error by -0.1307 $[-0.1395, -0.1213]$.

OULAD shows the same pattern outside our grading logs. In each of its nine course presentations, the three days before an assessment carry 1.29 to 2.95 times the activity of other days. With calendar features, a warning of peak activity seven days ahead reaches an F1 score (the harmonic mean of precision and recall) of 0.593, against 0.000 for a moving average of the previous two weeks.

7.3. Development Data, Sealed Data and Parameter Selection

Three logs have a part that was sealed before the method was designed: three academic terms of CodeBench (2023-1, 2023-2 and 2024-1), the last 20% of ACcoding's submission identifiers, and OULAD's 2014 year. Of the 809,330 sealed ACcoding rows, only the count and the first identifier have been read, and no result has been computed on the sealed parts of ACcoding and OULAD. Every access is recorded in the repository.

We opened the CodeBench terms once, on 25 September 2026, after the method, its parameters and the primary metric had been frozen, and Section 8.2 reports that evaluation. Every other number in this paper is a development number. A first attempt at the run stopped before producing any result, because a cleaning rule named only the development terms; the rule was extended to the sealed terms and the protocol frozen again (Supplementary Section S10).

A rule fixed before the grids were run chooses the guard's shape parameters. Each of the three shapes has a grid of parameter points, and the rule keeps the points whose worst harm over 15 validation cells is at most $G/2$. Among those it takes the one with the largest worst-cell gap closed (Supplementary Section S5). The validation cells are the five overlays at three loads, built on development terms other than the last.

However, the guard family and a coarse grid were explored on the evaluation overlays before this rule existed. The selection is therefore not held out from development, and we do not call any figure out of sample. Supplementary Section S5 records what the rule selected, which results the released code reproduces, and how the choice moves when one validation cell is dropped.

8. Experiments

Every policy is simulated on the shared pool of Section 7.2, with the same arrivals and costs, so each comparison is made job by job. Supplementary Table S4 lists the configuration.

8.1. The Headline: Scores That Use Only Available Results

The prediction study of Section 4 releases a job's result at the time the original platform recorded it. In the simulated queue, a job completes at a time that depends on the policy, so a score computed on the original clock may use a result that the simulated queue has not produced yet. The *online* protocol removes this for each copy of a class-term. It freezes the fitted model and simulates each policy once. When a job arrives, the model scores it from the results of earlier jobs in the same copy of its class-term that this simulation has completed by that moment, and from the recorded results of other classes and terms, which keep their original times. A score therefore follows the simulated queue

for its job's own copy and the recorded clock for all others. Supplementary Section S3.7 defines the simulation and checks it against a second implementation.

Four references stand beside the online protocol. The *exact* protocol also freezes the model. It simulates a policy, withholds every result that was not yet complete when the job using it arrived, rescores, and simulates again until no score uses an unavailable result. A result it has withheld once stays withheld, even when a later simulation completes it in time, so it can use less history than the online protocol. The *original* protocol keeps the platform's clock and is optimistic. The *conservative* protocol admits a result only an hour after it appeared, uses only history from the same class-term copy, and refits the models. The *static* protocol uses no outcome history at all. The guard's parameters are the same under all five.

At the busiest load, Guard(600) under the online protocol reaches a $p99_{dl}$ of 111.25 s and closes 0.705 [0.648, 0.768] of the gap (Table 4). The same guard closes 0.672 [0.602, 0.743] under the exact protocol, 0.801 [0.764, 0.842] under the original one, 0.429 [0.357, 0.533] under the conservative one and 0.349 [0.281, 0.457] under the static one. The online figure lies between the exact and the original ones. The exact protocol uses less history than the online one, and the original protocol uses results the queue has not produced. At the two lighter loads the two figures are closer, 0.725 against 0.740 and 0.800 against 0.837, and their intervals overlap. Unguarded SPJF-E closes 0.864 under the online protocol. The conservative figure falls further than the exact one because it also withholds history that the queue would have had and refits the models. Supplementary Section S3.7 separates the two effects. The parameters were selected under the original protocol. The full grids were run under the original protocol, so the rest of this section reports it unless stated otherwise, and its figures are optimistic references. Supplementary Section S11 replays 24 of the compared policies under the online protocol: the two unguarded rankings, the aging baseline, and at each promise the guard, its three single-shape winners, Fixed, Skip and Timeout. There every one of them closes less of the gap at the two higher loads and none with a promise exceeds it. Every paired difference that clears zero on the original clock keeps its sign, four no longer clear zero and three newly do, and the one conclusion of this section that changes is the choice between the two rankings in Section 8.3.

8.2. The Sealed Terms

The three sealed terms of the main log were opened once, on 25 September 2026, after the method, its parameters and the primary metric had been frozen (Section 7.3). The run replayed the frozen configuration and code and selected nothing, and the access ledger in the repository lists each of its ten commands and what each produced. The sealed terms hold fewer class-terms than the development pool, so each of their five overlays superposes 31 copies over 29 weeks. The number of servers follows the rule of Section 7.2, and the busiest hours reach utilisations of 0.487, 0.770 and 0.954 at the three loads.

At the busiest load, Guard(600) under the online protocol reaches a $p99_{dl}$ of 47.22 s and closes 0.894 [0.862, 0.912] of the gap (Table 5). It closes 0.880 under the exact protocol and 0.911 under the original one, and the five protocols fall in the same order as on the development terms. Unguarded SPJF-E closes 0.936 [0.909, 0.948] online. The guard's largest excess over FCFS is 533.8 s against its promise of 600 s, and its harm is 237.5 s, where SPJF-E reaches 3,301.6 s and 1,602.0 s. No guard in the main comparison exceeds its promise at any load (Supplementary Section S10).

Every policy closes a larger share of the gap on the sealed terms than on the development ones, except five of the deliberately corrupted rankings, which do more damage there. The sealed run therefore confirms the guarantee and the order of the policies, but not the size of the development figures. The running-time predictor ranks the sealed

Table 4. Guard(600) under five protocols for what the scores may use. Online: each score uses the results of its own class-term copy completed by its job's arrival in the simulated queue, and the recorded results of other classes and terms (the headline). Exact: results withheld and the queue simulated again until no score uses an unavailable result; a withheld result stays withheld. Original: the platform's own clock (optimistic). Conservative and static withhold more than the queue requires. Gap closed with 95% intervals from resampled weeks (Section 7.2), which do not cover the choice of classes and terms; waits, largest excess and harm in seconds. Fired: share of dispatch decisions, weighted by queue length, at which the guard overrode the base policy. At $\rho = 0.5$, $k = 7.8$ is the mean over the five overlays.

Load	Protocol	p99 _{dl}	Gap closed	Max excess	Harm	Fired (%)
$\rho = 0.5, k = 7.8$						
	online	15.22	0.725 [0.696, 0.746]	499.5	248.3	0.38
	exact	15.25	0.724 [0.697, 0.747]	498.6	248.1	0.45
	original	14.97	0.740 [0.713, 0.760]	492.2	237.0	0.17
	conservative	16.68	0.640 [0.604, 0.671]	499.9	235.2	0.94
	static	17.95	0.560 [0.519, 0.594]	498.0	225.1	0.38
$\rho = 0.8, k = 5$						
	online	50.57	0.800 [0.772, 0.819]	509.3	215.7	2.18
	exact	51.90	0.785 [0.748, 0.807]	509.0	215.6	2.64
	original	47.47	0.837 [0.815, 0.851]	507.8	319.6	1.31
	conservative	64.77	0.634 [0.560, 0.688]	509.4	327.7	3.26
	static	70.18	0.571 [0.504, 0.638]	512.9	341.6	3.89
$\rho = 1, k = 4$						
	online	111.25	0.705 [0.648, 0.768]	517.7	333.2	7.27
	exact	118.94	0.672 [0.602, 0.743]	519.9	332.8	7.82
	original	89.24	0.801 [0.764, 0.842]	523.7	339.9	7.15
	conservative	174.71	0.429 [0.357, 0.533]	527.8	303.5	7.71
	static	193.11	0.349 [0.281, 0.457]	528.1	316.1	11.18

terms as well as the development ones, with a pooled AUROC of 0.9414 for SPJF-E's score against 0.9360 on the development terms.

At the busiest load the guard closes 0.189 more of the gap on the sealed terms under the online protocol. Unguarded SPJF-E closes 0.072 more, and the guard's cost against SPJF-E falls from 0.158 to 0.041, which gives the other 0.117. The load on the two pools does not account for the difference. One cell from each pool has four servers and a busiest-hour utilisation of 0.980 and 0.979, and FCFS and SJF differ by 220.1 s and 223.6 s in p99_{dl} in these two cells; the guard closes 0.681 of the gap in the first and 0.889 in the second. The larger share on the sealed terms is confined to the primary metric. The jobs that this metric reads, those arriving in the 24 hours before a deadline, are 37.5% of the sealed jobs against 23.9% of the development ones. On the 99th-percentile wait over all jobs, the same ratio gives the guard 0.904 and 0.896 and SPJF-E 0.956 and 0.932, and on the mean wait SPJF-E closes less on the sealed terms at all three loads (0.819 against 0.903 at the busiest).

8.3. What to Rank By

On the original clock, ranking by expected cost beats the log-scale score at the two higher loads and ties with it at the lightest. The paired difference in gap closed is -0.000 $[-0.016, 0.013]$, $+0.012$ $[0.004, 0.019]$ and $+0.019$ $[0.011, 0.024]$ at $\rho = 0.5, 0.8$ and 1.0 . Refitting the log-scale model from scratch moves this difference by at most 0.001 (Supplementary Table S16). The online protocol does not confirm this order. There the three differences are -0.022 $[-0.039, -0.009]$, $+0.006$ $[-0.004, 0.015]$ and $+0.009$ $[-0.000, 0.015]$, so the log-scale score is ahead at the lightest load and the two tie at the higher ones (Supplementary Table S32). Both scores are replayed with their own frozen models, so the change comes from the history each may use. The choice of SPJF-E as the base ranking therefore rests

Table 5. Guard(600) under the five protocols of Table 4 on the three sealed terms, opened once after the method and its parameters were frozen. Columns as in Table 4; k is the mean over the five overlays.

Load	Protocol	p99 _{dl}	Gap closed	Max exc.	Harm	Fired (%)
$\rho = 0.5, k = 8.2$						
	online	3.76	0.895 [0.872, 0.906]	260.8	166.2	2.59
	exact	3.77	0.893 [0.871, 0.903]	256.6	166.6	3.29
	original	3.75	0.897 [0.878, 0.907]	264.0	173.9	4.01
	conservative	4.15	0.839 [0.816, 0.858]	264.1	178.0	3.40
	static	4.46	0.792 [0.744, 0.824]	259.3	203.3	0.16
$\rho = 0.8, k = 5.2$						
	online	24.59	0.884 [0.862, 0.903]	506.6	202.6	6.40
	exact	24.74	0.880 [0.860, 0.899]	499.2	202.6	4.41
	original	24.22	0.891 [0.874, 0.908]	504.6	189.5	7.01
	conservative	28.09	0.811 [0.772, 0.845]	520.3	233.2	7.06
	static	31.70	0.740 [0.691, 0.786]	533.7	255.3	1.33
$\rho = 1, k = 4.2$						
	online	47.22	0.894 [0.862, 0.912]	533.8	237.5	7.65
	exact	49.40	0.880 [0.839, 0.903]	526.5	243.1	5.25
	original	44.33	0.911 [0.890, 0.926]	522.9	293.2	7.10
	conservative	65.08	0.784 [0.712, 0.845]	518.8	244.6	8.57
	static	82.66	0.679 [0.583, 0.783]	528.6	332.3	3.20

on the original protocol. On the original clock, quantile scores and squared error on raw seconds lose to the Tweedie fit at every load. A rule that only separates predicted-heavy from predicted-light jobs recovers little of the benefit, so most of the benefit comes from the order inside the light class (Supplementary Table S14).

The two kinds of prediction error do not cost the same. Replacing the largest 1% of underestimates by the true cost closes 0.91 of the distance to SJF in p99_{dl} at $\rho = 1.0$ on both overlays tested. Repairing the largest 1% of overestimates closes only 0.26 and 0.24. At 0.1%, the repair of underestimates closes 0.47 and that of overestimates 0.04. Treating a long job as short is thus the expensive mistake, by about an order of magnitude. The repair depends on the score's numerical scale, so this result describes our fitted predictor and not prediction errors in general.

8.4. What the Guarantee Costs and Gains

Table 6 gives the guard at three bounds and three loads. At $\rho = 1.0$, Guard(600) closes 0.801 of the gap and the unguarded policy 0.916. A bound of ten time limits therefore costs 0.115 of the gap. The cost is 0.072 at $G = 1200$ s and 0.438 at $G = 300$ s. The paired intervals of these three differences are [-0.142, -0.077], [-0.083, -0.052] and [-0.526, -0.314], and none contains zero, so the guarantee has a cost at every G we tested. The cost falls as the slack above the floor of Corollary 8, $G - (3 - 2/k)L$, grows against FCFS's p99_{dl} of 273.50 s: the slack is 0.55, 1.65 and 3.84 times that wait at $G = 300, 600$ and 1200 s. Under the online protocol the cost is larger, 0.158 [0.110, 0.191], 0.110 [0.082, 0.124] and 0.497 [0.393, 0.565] (Supplementary Table S32). What the guarantee buys is protection of the worst job. The unguarded policy delays one job at the busiest load by 5,904.5 s beyond its FCFS wait, 98.4 times the 60-second limit. Exact SJF, which knows every running time, delays one job by 4,755.5 s, 79.3 limits, so most of this delay comes from ordering by length and not from wrong predictions. The guard keeps every job within its bound. It also keeps the worst wait among heavy jobs within about 450 s of its FCFS value, whereas the unguarded policy multiplies that wait by 2.8 to 4.1. The price of this protection is paid in the mean and just below the worst job. At the busiest load the guard keeps 0.751 of SJF's reduction of the

mean wait, against 0.930 unguarded, and its 99.9th-percentile wait over all jobs is 717.6 s, against 192.4 s unguarded and 984.3 s under FCFS.

Table 6. Guard(G) at three bounds and three loads, original protocol, with the FCFS and SPJF-E figures of each load in its header. Gap closed with 95% intervals. Reduction is the plain percentage reduction of $p_{99_{dl}}$ against FCFS, which is a different number. Largest excess and harm are the worst over the five overlays, and the other columns are means. Times in seconds. Fired as in Table 4. Supplementary Table S6 adds the comparators.

Load	Policy	G (s)	$p_{99_{dl}}$	Gap closed	Reduction (%)	Max excess	Harm	Fired (%)
$\rho = 0.5, k = 7.8$; FCFS $p_{99_{dl}} = 27.64$ s, SPJF-E 14.68 s with max excess 1,241.4 s								
	Guard(300)	300	15.58	0.705 [0.678, 0.724]	43.7	192.5	132.1	1.35
	Guard(600)	600	14.97	0.740 [0.713, 0.760]	45.8	492.2	237.0	0.17
	Guard(1200)	1200	14.78	0.750 [0.723, 0.772]	46.4	996.7	106.0	0.09
$\rho = 0.8, k = 5$; FCFS $p_{99_{dl}} = 119.35$ s, SPJF-E 43.33 s with max excess 3,044.5 s								
	Guard(300)	300	55.94	0.738 [0.697, 0.761]	53.0	209.7	114.1	11.57
	Guard(600)	600	47.47	0.837 [0.815, 0.851]	60.1	507.8	319.6	1.31
	Guard(1200)	1200	46.31	0.850 [0.829, 0.865]	61.1	1,096.1	117.7	0.41
$\rho = 1.0, k = 4$; FCFS $p_{99_{dl}} = 273.50$ s, SPJF-E 62.79 s with max excess 5,904.5 s								
	Guard(300)	300	163.40	0.478 [0.387, 0.602]	40.2	229.1	134.8	28.64
	Guard(600)	600	89.24	0.801 [0.764, 0.842]	67.3	523.7	339.9	7.15
	Guard(1200)	1200	79.32	0.844 [0.817, 0.870]	70.9	1,110.4	123.4	1.40

The harm limit used in selection does not always carry over to the evaluation overlays. Guard(600) has harms of 319.6 and 339.9 s at the two higher loads, above the 300 s validation limit, although every excess stays within the bound. The shape of the budget matters mainly where the bound is tight and the load high. There it decides which jobs are delayed more than how much of the benefit survives (Supplementary Sections S3.3 and S3.11). Counting overtakes instead of work gains almost nothing on this load (Supplementary Figure S2). Overtakers here are mostly shorter than a second, while a count must be sized against the worst case of L per pass. Our count baseline is charged when an overtaker starts. Charging it at completion instead changes its gap closed by at most 0.015, and by at most 0.001 at the two higher loads. An aging baseline ranks by $\hat{C}_i - \omega(t - a_i)$, with the credit ω chosen under the same harm limit. At the busiest load under the conservative protocol it closes 0.051 of the gap, against 0.429 for the guard, and one job exceeds its FCFS wait by 855.6 s. Under the online protocol it closes 0.258 against 0.705, and one job exceeds its FCFS wait by 1,808.9 s. Aging carries no proved bound (Supplementary Tables S12 and S31).

A maximum waiting time gives a guarantee of the same form (Proposition 10), so the guard's counter has to earn its place against it. Table 7 runs Timeout($G - (3 - 2/k)L$) beside Guard(G), which gives both the bound G . At $\rho = 0.5$ the two differ by at most 0.053 of the gap. Near saturation they part. At $\rho = 1.0$ and $G = 600$ s the clock closes 0.420 and the guard 0.801. At $G = 300$ s the clock closes 0.031, and its $p_{99_{dl}}$ of 266.4 s is barely below FCFS's 273.5 s. At $G = 1200$ s the two agree again. The clock's bound is not known to be tight, so we also compare the two at equal realised worst excess. On the first overlay at the busiest load, Timeout with $\theta = 480$ s reaches a largest excess of 506.1 s and closes 0.490 of the gap. Guard(600) reaches 512.3 s and closes 0.797. The clock also harms more, 456.7 s against 339.9 s at $\rho = 1.0$ and $G = 600$ s. Under the online protocol every difference of Table 7 keeps its sign (Supplementary Table S33). At $\rho = 1.0$ and $G = 600$ s the clock closes 0.282 and the guard 0.705, a paired difference of $+0.423 [+0.204, +0.664]$, and at $G = 300$ s the clock closes 0.023.

Ordering by predicted cost inside fixed windows, as in the closest scheduler for judging queues [53], limits overtaking by position. We ran a version of it on the first overlay with windows of up to 512 jobs. It closes at most 0.417, 0.146 and 0.057 of the gap at

Table 7. Guard(G) against the maximum waiting time $\text{Timeout}(G - (3 - 2/k)L)$, which has the same bound G. Gap closed, mean over the five overlays, with 95% intervals. The difference is Guard minus Timeout, resampled as a difference. Harm is the worst over the overlays, in seconds. Original protocol.

Load	G (s)	Guard(G)	Timeout	Difference	Harm, Guard / Timeout
0.5	300	0.705 [0.678, 0.724]	0.652 [0.619, 0.679]	+0.053 [+0.030, +0.077]	132.1 / 148.0
	600	0.740 [0.713, 0.760]	0.734 [0.707, 0.753]	+0.006 [+0.001, +0.012]	237.0 / 237.0
	1200	0.750 [0.723, 0.772]	0.752 [0.724, 0.774]	−0.002 [−0.005, +0.001]	106.0 / 237.0
0.8	300	0.738 [0.697, 0.761]	0.244 [0.017, 0.508]	+0.494 [+0.241, +0.691]	114.1 / 177.1
	600	0.837 [0.815, 0.851]	0.789 [0.754, 0.807]	+0.048 [+0.030, +0.080]	319.6 / 443.7
	1200	0.850 [0.829, 0.865]	0.857 [0.837, 0.869]	−0.006 [−0.012, −0.001]	117.7 / 469.2
1.0	300	0.478 [0.387, 0.602]	0.031 [0.012, 0.055]	+0.447 [+0.369, +0.550]	134.8 / 182.3
	600	0.801 [0.764, 0.842]	0.420 [0.131, 0.673]	+0.381 [+0.158, +0.650]	339.9 / 456.7
	1200	0.844 [0.817, 0.870]	0.827 [0.786, 0.865]	+0.017 [−0.002, +0.044]	123.4 / 708.8

the three loads, with largest excesses of 232.7, 359.6 and 444.5 s. On the same overlay Guard(300) closes 0.678, 0.716 and 0.486, with 189.3, 199.7 and 219.6 s. Near saturation, a window large enough to help lets long jobs wait longer than the guard does. The last test corrupts the scores deliberately, by reversing them, by drawing them at random, or by calling the longest 1% of jobs the shortest. The unguarded policy then delays some job by more than 6,000 s beyond FCFS at the busiest load, and Guard(600) keeps every job within its bound. The guarded $p_{99_{dl}}$ is then 546.5, 261.9 and 543.6 s, against FCFS's 273.50 s. Over these three corrupted scores and the five protocols of Section 8.1, Guard(600) closes between −1.187 and 0.801 of the gap at the busiest load, while its largest excess over FCFS stays between 517.7 and 534.8 s. The guarantee thus protects the worst single job but not the shape of the tail, and a guarded run may have a worse percentile than FCFS, by at most G (Supplementary Table S8).

8.5. The Bounds on the Traces

Every run that includes a bounded policy checks the bound on every job inside the simulation, so a violation would expose an error in the code rather than in the theorem. The 315 policy-by-cell runs reported here make 5,554,949,400 such checks. The single-server runs add 67,396,287, and an independently written second simulator adds 5.7 billion. None of them finds a violation, and the count of violations is 0. Proposition 10 was checked job by job on the same overlays and Proposition 11 on all 304 bounded runs, also with 0 violations. Proposition 12 was checked on constructed instances (Supplementary Section S4.12). The ratio of the realised worst excess to the bound falls as k grows, from 0.9950 at $k = 1$ to 0.8850 at $k = 8$. At one server Guard(600) reaches 595.5 s of its 600 s bound (Supplementary Section S3.6). The residual Δ_i of Theorem 2 never exceeds 0.518 of its own bound $2(k - 1)L$ (Supplementary Sections S3.13 and S3.16).

The absolute bound of Proposition 11 comes close to the realised waits. At one server every FCFS wait equals its fluid backlog to the microsecond, as the proposition predicts, and the largest wait reaches 0.999 of the bound. At the three loads the worst wait under a work budget reaches 0.886, 0.951 and 0.963 of the bound, and under FCFS 0.881, 0.945 and 0.965. An absolute bound quoted in advance needs a forecast of the burst σ . The burst of the validation terms predicts that of the evaluation terms only to within a factor of 0.738 to 1.573, and it is too low in 10 of 15 cells. A bound quoted from it is broken in 18 of 60 runs, by up to 349.8 s. Such a bound therefore needs a margin, here a factor of 1.36 on σ , or an admission control that keeps the arrivals within the quoted burst.

8.6. Where the Method Applies

Table 8 places the main log beside the other workloads. Excesses are in units of each log's L , the unit in which the guarantee is written. For the grading logs L is the time limit, and for the others it is the analysis cap of Section 7.1. Two screens say where the guard is useful. The first compares the FCFS tail with the guarantee's additive term $(3 - 2/k)L$, which is close to $3L$. If the FCFS p99 is below this term, the guarantee is coarser than the waits themselves. CodeBench, at $4.56L$, and the cloud-function log, at $4.22L$, clear the first screen, and four of the six workloads do not. The compute farm is the extreme case, with an FCFS p99 of 18.7 h against $L = 493,857$ s, or 5.7 days. The second screen measures how concentrated the costs are, which is the opportunity for reordering. In the two CI pools the longest 1% of jobs carry only 3.4% and 5.2% of the work. There, ordering by true cost makes the p99 worse in three of the four windows we examined. Where both screens are cleared, the guard closes 0.48 to 0.71 of the gap and holds the largest excess to 2.5 to $3.8L$ at a bound of $5L$. Clearing both screens does not guarantee a gain, as the next subsection shows.

Table 8. The workloads under one format. Tail share: fraction of executed work carried by the longest 1% of jobs. AUROC: heavy label from information available at arrival. Screen 1: whether the FCFS p99 exceeds $(3 - 2/k)L$. Excess in units of each log's L . A negative gap closed means that the policy's p99 is worse than FCFS's. CodeBench row: original protocol. Supplementary Section S6.7 describes the CI capacities and adds a log from a grid computing cluster.

Log and load	Tail share	AUROC	FCFS p99/ L	Screen 1	Gap closed, SPJF-E	Unguarded max excess	Guard at $G \approx 5L$ gap closed / max excess
CodeBench, $\rho \approx 1.0$	0.375	0.936	4.56	yes	0.916	98.4L	0.478 / 3.82L
ACcoding, drawn arrivals, $\rho = 0.8^a$	0.122	0.927	0.021	no	0.980	20L	0.974 / 2.21L
Azure Functions, $\rho = 1.0$	0.443	0.985	4.22	yes	0.858	10.07L	0.710 / 2.52L
CI pool A, $k = 156$ of 173	0.034	0.651	1.78	no	n/a ^b	11.53L	0.735 (mean) ^b / 2.64L
CI pool B, $k = 96$ of 105	0.052	0.997	0.373	no	0.958	0.34L	0.958 / 0.34L
Intel Netbatch, $\rho = 0.95$	0.372	0.890	0.136	no	-4.111	0.85L	-4.346 / 0.83L

^a ACcoding arrivals are drawn, so this row describes the ordering and not waiting on that platform. Its budget is the largest tested, $B = 10L$, and its tail share is simulated with $L = 30.5$ s (0.275 on measured cost at $L = 30.0$ s, the figure of Section 1). ^b On CI pool A, SJF is itself worse than FCFS at the p99, so no p99 gap is defined; the guard's entry is the gap in mean wait and is not comparable with the other rows.

8.7. Capacity, Physical Runs and Feedback

The benefit depends on capacity. We ran every k from 1 to 20 on the five overlays. At $k = 1$, Guard(600) makes p99_{dl} worse by 493.79 s, and all five overlays agree that it improves the p99 only for $k = 4$ to 12. We also ran the guard physically, on two worker processes that occupy real time slots for each job's recorded duration. This run executed 2,151 jobs of one development window, and all 6,453 dispatch decisions followed the rule. Against FCFS on the same run's measured arrivals and running times, every job stayed within the promise of Guard(300), with a largest excess of 222.21 s. A physical run needs the bound widened by the time workers sat idle during dispatch, at most 1.127 s here, and no job came close to needing it. Both processes ran on one machine, so this run tests the dispatch rule, not network delay. The guarded p99 of 1,783.65 s was nevertheless no better than FCFS's 1,763.30 s, and single jobs differed from the ideal simulation by up to 1,354.10 s. A second physical test used the two-worker back end of a course platform, where twenty scripted accounts submitted 100 jobs per cell in a deadline-shaped burst with $G = 400$ s. Unguarded predicted order left 7 jobs beyond the bound. The four guarded cells left none, with a worst excess of 0.514G (Supplementary Sections S6.1, S6.2 and S6.9).

The simulations replay arrivals at their recorded times, so they neither reward a scheduler with the work it would attract nor punish it for the users it would drive away [78,79]. The logs record one part of that feedback, the time between one result and the

same student's next submission. In a second replay, each submission is released only after its predecessor's result plus that recorded delay. This replay shortens $p99_{dl}$ by a factor of 1.3 to 3.8. Guard(600) then closes 0.825, 0.809 and 0.810 of the gap at the three loads, against 0.734, 0.827 and 0.800 with the recorded arrival times, and it keeps every job within its bound (Supplementary Section S6.5).

9. Limitations

The queue is constructed, and the replay is open-loop. CodeBench gives each student a separate container [77], so its logs do not record waiting in a shared queue [21]. We use these logs as a record of demand, with reconstructed arrival times and recorded running times. The pool, the queue and the scheduler are ours, and nothing here describes waiting on that platform. A fixed pool of grading machines behind one queue is a standard deployment [7,8], and on the same demand 14 servers replace 1,636 concurrent containers [80] (Supplementary Section S2.3). Arrivals are replayed at their recorded times, so a scheduler is neither rewarded with the work it would attract nor punished for the users it would drive away. Replaying each student's recorded delay between one result and the next submission (Section 8.7) restores one part of that feedback and does not reverse the benefit. It restores neither abandonment nor changes in what students submit, however, and the recorded delays come from a platform on which no student waited. Our results are therefore conditional on a fixed amount of offered work, and Supplementary Section S6.3 constructs feedback under which a scheduling advantage disappears or reverses. We raise the load by overlaying whole class-terms aligned on weekday and clock time, rather than by compressing the gaps between arrivals [81], which would also compress the daily cycle [78].

The guarantee is written in units of L , and per job. Every additive constant in Section 6 is a multiple of the time limit, apart from the report delay δ and the arrival burst σ , and the guard's constant $(3 - 2/k)L$ cannot be improved. No wrapper that reorders at all can guarantee less than L around every base policy (Proposition 13). Stopping long jobs and restarting them in a second tier, as task assignment by guessing size does [82], does not escape this floor on a shared pool. The stopped work is lost rather than overtaken, and the second tier is again non-preemptive in blocks of L (Supplementary Section S7.5). Charging a cap known at arrival lowers the constant by exactly L and leaves the floor. Because the guarantee bounds each job, it also bounds each percentile, $Q_\alpha(W_P) \leq Q_\alpha(W_{FCFS}) + G$. It does not ensure that a percentile improves, and under corrupted scores a guarded run had a worse $p99_{dl}$ than FCFS (Section 8.4). Where the FCFS tail is already below about $3L$, the bound is coarser than the waits it bounds, which is the case for four of the six workloads (Section 8.6).

Development results and the sealed terms. We explored the guard family and an early parameter grid on the overlay the tables report, and only then searched the pre-stated grids on validation overlays that exclude the evaluation term. The selection is thus validation-only within a family that is not. The worst-cell rule is sensitive to which cells it sees, and the five overlays are week-shifts of the same class-terms rather than independent samples. Resampling whole class-terms was run under the original protocol only, so the online figures carry intervals from resampled weeks, which do not cover the choice of classes and terms. Moreover, the shape selected on 15 cells with $k \geq 4$ does not carry over to a single server (Supplementary Section S3.14). We therefore do not call the parameters validated. The guarantee does not depend on them, and only the benefit does. The sealed terms were opened once, after the method, parameters and metric were frozen (Section 8.2). They come from the same platform and pass through the same constructed queue, so they test the frozen choices on new classes and terms but not on another service.

Information, costs, and what was not studied. The guard's parameters were selected under the original protocol. Of the comparisons after Section 8.1, only the 24 main policies of Sections 8.3 and 8.4 were also replayed under the online one (Supplementary Section S11). Every other comparison there, and the queue comparison that chose the predictor (Section 4.3), rests on the optimistic original protocol. The online protocol takes the results of other classes and terms at their recorded times. The recorded costs are wall-clock durations that already contain some contention, and the 60-second limit is inferred from the data rather than documented. ACcoding arrival times are drawn because the released tables omit them (Supplementary Section S2.1). Preemption is outside the model. The wrapper applies unchanged to a learned or reinforcement-learned base policy, which we did not test. Capacity is fixed within each replay, and the capacity sweep is a sensitivity, not a planning study. Computing the features and running the predictor are charged only in a sensitivity run, where they move the gap closed by less than 0.001 (Section 4). We ran a physical service but no production deployment, and we measured no user response. Nor did we combine the guard with an existing scheduler such as the serverless prototype of Fu et al. [59], which has no proved per-job bound.

10. Conclusions

A service that runs many users' jobs on a few shared servers can order its queue by predicted running time and still guarantee every job a wait no more than G seconds longer than in arrival order, whatever the predictions are. This guarantee follows from one identity. A job's extra wait relative to FCFS equals the net running time of the jobs that overtook it, divided by the number of servers, up to at most $(2 - 2/k)L$. Limiting that overtaken work is therefore sufficient for a per-job guarantee and, up to an additive $O(kL)$, necessary. The guard enforces such a limit with the exact constant $(3 - 2/k)L$, and its guarantee survives completion reports that arrive up to δ late at a cost of $L + \delta$. A maximum waiting time gives a guarantee of the same form. Near saturation and at tight bounds, however, it keeps much less of the benefit, because it charges a job for all the work ahead of it and not only for the jobs that pass it. Counting overtakes gives the same guarantee only at a charge of L per pass, which on the main log is 150 times the mean running time.

For the grading system that opened this paper, the answer to a student whose submission sat behind a looping one is now a number. At the busiest load of the recorded course demand, with every score computed from the results that its job's own copy of the demand had returned when the job arrived, a guard with $G = 600$ s keeps 0.705 of the reduction in 99th-percentile wait before deadlines that SJF achieves over FCFS. Unguarded ordering keeps 0.864, and with the guard no job exceeds its bound. On three terms held back until the method was frozen, the same guard keeps 0.894 and unguarded ordering 0.936. In seconds, the 99th-percentile wait of the submissions made in the 24 hours before a deadline falls from 273.50 s under FCFS to 111.25 s with the guard, and from 187.20 s to 47.22 s on the held-back terms. No job waits more than 600 s beyond its FCFS wait, where unguarded ordering delayed one job by 5,812.7 s. The scores come from gradient-boosted trees on each student's and each task's history, chosen on queueing outcomes rather than prediction accuracy. A heterogeneous graph network over students, tasks, assignments and cohorts has a rank correlation between the two tree models' over all 40,844 test submissions and was not run through the queue. On the 1,092 submissions to tasks with fewer than five earlier results, the situation that a newly released assignment creates, it ranks jobs better than the trees by 0.146 in rank correlation, and the same network without edges does so by 0.142, so the gain there comes from the neural model of each job's

own features. The demand is constructed in both evaluations, and the benefit depends on capacity: it appeared only for $k = 4$ to 12.

Two questions remain open. Under the executed-work convention of Section 6.1, the lower constant of the identity is known only to lie in $[(k-1)L, 2(k-1)L]$. We also do not know whether some wrapper overrides the base policy strictly less often than Algorithm 1 while guaranteeing the same G .

Supplementary Materials: The following supporting information can be downloaded at <https://www.mdpi.com/article/10.3390/math1010000/s1>: Table S1: Predictors on the non-judging logs, by feature group; Table S2: Predictors on the two grading logs under one protocol; Table S3: Share of work carried by the longest jobs, and the provenance of L ; Table S4: Experimental configuration and selection protocol; Table S5: Selected budget points and per-shape grid winners; Table S6: The guard at three bounds, with the comparators; Table S7: The age-relative and queue-length winners; Table S8: Deliberately corrupted scores; Table S9: The residual of Theorem 2 on nine cells; Table S10: The single-server configuration; Table S11: Definition of linear aging; Table S12: Comparison with the aging baseline; Table S13: Predictor AUROC with user-block intervals; Table S14: Alternative ranking scores; Table S15: Trace-construction sensitivities; Table S16: Unguarded policies; Table S17: Candidates and feasible points of the selection; Table S18: Whole weeks against whole class-terms as the sampling unit; Table S19: Capacity sweep; Table S20: Physical timed-work runs; Table S21: Replay with recorded think times; Table S22: A grid log by walltime class; Table S23: The grid log's 900-second class at its own limit; Table S24: The grid-log rows of the cross-log comparison; Table S25: Six cells on a course platform's back end; Table S26: Scope of the nearest per-job guarantees; Table S27: Sealed terms: ranking scores; Table S28: Sealed terms: the guard and the comparators; Table S29: Sealed terms: corrupted scores; Table S30: Sealed terms: a single server; Table S31: The main policies under the online replay; Table S32: Paired differences on both clocks; Table S33: The guard against a maximum waiting time under the online replay; Figure S1: Gap-harm trade-offs of the budget shapes; Figure S2: What each budget shape keeps as the bound is relaxed; Figure S3: Original-score reference and corrupted-score controls. The Supplementary Materials also give the proofs deferred from Section 6 (Sections S4, S7 and S8), extended related work (Section S9), the sealed terms (Section S10) and the comparisons under the online replay (Section S11).

Author Contributions: Conceptualization, [A.B.] and [C.D.]; methodology, [A.B.]; software, [A.B.]; validation, [A.B.] and [C.D.]; formal analysis, [A.B.]; investigation, [A.B.]; resources, [C.D.]; data curation, [A.B.]; writing—original draft preparation, [A.B.]; writing—review and editing, [A.B.] and [C.D.]; visualization, [A.B.]; supervision, [C.D.]; project administration, [C.D.]; funding acquisition, [C.D.]. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Nanjing University of Posts and Telecommunications, grant number XC2025021.

Institutional Review Board Statement: Not applicable. This study used previously published, pseudonymised logs and involved no human participants and no animals.

Informed Consent Statement: Not applicable.

Data Availability Statement: The public source datasets used here are available at the locations cited, and we redistribute none of them. The CodeBench judging logs are published by the Universidade Federal do Amazonas [21]; the ACcoding dataset is published with its data descriptor [22]; the OULAD is published with its own [76]; the Azure Functions 2021 invocation trace is published in the Azure Public Dataset repository [5], which asks that [6] be cited; and the Intel Netbatch logs are published in the Parallel Workloads Archive [25], which asks that the logs be acknowledged to their depositors and suggests citing [26]. The LPC-EGEE log is also published in the Parallel Workloads Archive [83], with acknowledgment requested for its data provider. The CI trace of the two Firefox hardware worker pools, was collected by us from Mozilla's public, unauthenticated Taskcluster and Treeherder application programming interfaces [23,24]; we do not redistribute it, and the collection scripts that rebuild it from those interfaces are part of the released code. The Azure invocation trace

is distributed under the publisher’s Creative Commons Attribution licence [5]; public availability does not imply an absence of licence conditions. No restricted access agreement was used, and no personal data were processed beyond the pseudonymous user identifiers the published judging datasets already carry. The simulator, the predictors, the guard implementation, configurations and archived development analyses are available at <https://github.com/oscar030406/spjf-guard>. Supplementary Section S5 distinguishes the tables reproduced by the package from the earlier development evidence.

Acknowledgments: During the preparation of this manuscript, the authors used Claude (Anthropic; Claude Code with Claude Opus models) and Codex (OpenAI) for drafting and editing text, writing and reviewing the simulation and analysis code, searching the literature, and searching for counterexamples to the stated propositions. The authors have reviewed and edited the output and take full responsibility for the content of this publication.

Conflicts of Interest: The authors declare no conflicts of interest. [To be confirmed by the authors: if there is a funder, state here what role it had in the design of the study, in the collection, analysis or interpretation of data, in the writing of the manuscript, or in the decision to publish the results.]

Abbreviations

The following abbreviations are used in this manuscript:

AUROC	Area under the receiver operating characteristic curve
CI	Continuous integration
EASY	Extensible Argonne Scheduling sYstem
FCFS	First-come first-served
GNN	Graph neural network
GRU	Gated recurrent unit
OULAD	Open University Learning Analytics Dataset
p99	99th percentile
R-GCN	Relational graph convolutional network
RMSE	Root mean squared error
SJF	Shortest job first
SOAP	Schedule Ordered by Age-based Priority
SPJF-E	Shortest predicted job first on the expected-cost score
SPJF-log	Shortest predicted job first on the log-scale score
SRPT	Shortest remaining processing time

References

- Wasik, S.; Antczak, M.; Badura, J.; Laskowski, A.; Sternal, T. A Survey on Online Judge Systems and Their Applications. *ACM Computing Surveys* **2018**, *51*, 1–34. <https://doi.org/10.1145/3143560>.
- König, C.J.; Kleinmann, M. Deadline Rush: A Time Management Phenomenon and Its Mathematical Description. *The Journal of Psychology* **2005**, *139*, 33–45. <https://doi.org/10.3200/JRLP.139.1.33-45>.
- Alfi, V.; Parisi, G.; Pietronero, L. Conference registration: how people react to a deadline. *Nature Physics* **2007**, *3*, 746. <https://doi.org/10.1038/nphys761>.
- Alfi, V.; Gabrielli, A.; Pietronero, L. How people react to a deadline: time distribution of conference registrations and fee payments. *Open Physics* **2009**, *7*, 483–489. <https://doi.org/10.2478/s11534-009-0059-z>.
- Microsoft Azure. Azure Functions Invocation Trace 2021. <https://github.com/Azure/AzurePublicDataset/blob/master/AzureFunctionsInvocationTrace2021.md>, 2021.
- Zhang, Y.; Goiri, Í.; Chaudhry, G.I.; Fonseca, R.; Elnikety, S.; Delimitrou, C.; Bianchini, R. Faster and Cheaper Serverless Computing on Harvested Resources. In Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP ’21), 2021, pp. 724–739. <https://doi.org/10.1145/3477132.3483580>.
- Peveler, M.; Maicus, E.; Cutler, B. Comparing Jailed Sandboxes vs Containers Within an Autograding System. In Proceedings of the 50th ACM Technical Symposium on Computer Science Education (SIGCSE ’19), 2019, pp. 139–145. <https://doi.org/10.1145/3287324.3287507>.

8. DOMjudge team. DOMjudge Administrator's Manual: Overview and Judging. <https://www.domjudge.org/docs/manual/main/overview.html>. Accessed 2026-09-19.
9. Lykouris, T.; Vassilvitskii, S. Competitive Caching with Machine Learned Advice. *Journal of the ACM* **2021**, *68*, 1–25. Preliminary version in Proceedings of the 35th International Conference on Machine Learning (ICML 2018), PMLR 80:3296–3305; preprint arXiv:1802.05399, <https://doi.org/10.1145/3447579>.
10. Purohit, M.; Svitkina, Z.; Kumar, R. Improving Online Algorithms via ML Predictions. In Proceedings of the Advances in Neural Information Processing Systems 31 (NeurIPS 2018), 2018.
11. Im, S.; Kumar, R.; Montazer Qaem, M.; Purohit, M. Non-Clairvoyant Scheduling with Predictions. In Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '21), 2021, pp. 285–294. <https://doi.org/10.1145/3409964.3461790>.
12. Lattanzi, S.; Lavastida, T.; Moseley, B.; Vassilvitskii, S. Online Scheduling via Learned Weights. In Proceedings of the 2020 ACM-SIAM Symposium on Discrete Algorithms (SODA), 2020, pp. 1859–1877. <https://doi.org/10.1137/1.9781611975994.114>.
13. Lindermayr, A.; Megow, N. Permutation Predictions for Non-Clairvoyant Scheduling. In Proceedings of the 34th ACM Symposium on Parallelism in Algorithms and Architectures (SPAA '22), 2022, pp. 357–368. Preprint arXiv:2202.10199, <https://doi.org/10.1145/3490148.3538579>.
14. Scully, Z.; Grosz, I.; Mitzenmacher, M. Uniform Bounds for Scheduling with Job Size Estimates. In Proceedings of the 13th Innovations in Theoretical Computer Science Conference (ITCS 2022). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, Vol. 215, *LIPICs*, pp. 114:1–114:30. Preprint arXiv:2110.00633, <https://doi.org/10.4230/LIPICs.ITCS.2022.114>.
15. Mitzenmacher, M.; Shahout, R. Queueing, Predictions, and Large Language Models: Challenges and Open Problems. *Stochastic Systems* **2025**, *15*, 195–219. Preprint arXiv:2503.07545, <https://doi.org/10.1287/stsy.2025.0106>.
16. Zhao, T.; Xia, C.; Chang, X.; Li, C.; Li, W.; Zomaya, A. Competitive Fair Scheduling with Predictions. In Proceedings of the The Thirteenth International Conference on Learning Representations (ICLR 2025), 2025.
17. Yu, G.; Scully, Z. Strongly Tail-Optimal Scheduling in the Light-Tailed M/G/1. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2024**, *8*, 1–33. Preprint arXiv:2404.08826, <https://doi.org/10.1145/3656011>.
18. Yu, G.; Harlev, A.; Adakroy, R.; Scully, Z. A Tale of Two Traffics: Optimizing Tail Latency in the Light-Tailed M/G/k. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2025**, *9*, 1–40. <https://doi.org/10.1145/3771561>.
19. Ke, G.; Meng, Q.; Finley, T.; Wang, T.; Chen, W.; Ma, W.; Ye, Q.; Liu, T.Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In Proceedings of the Advances in Neural Information Processing Systems 30 (NeurIPS 2017), 2017, pp. 3146–3154.
20. Schlichtkrull, M.; Kipf, T.N.; Bloem, P.; van den Berg, R.; Titov, I.; Welling, M. Modeling Relational Data with Graph Convolutional Networks. In Proceedings of the The Semantic Web (ESWC 2018), 2018, Vol. 10843, *Lecture Notes in Computer Science*, pp. 593–607. Preprint arXiv:1703.06103, https://doi.org/10.1007/978-3-319-93417-4_38.
21. Universidade Federal do Amazonas, Instituto de Computação. CodeBench Dataset, version 1.81. <https://codebench.icomp.ufam.edu.br/dataset/>. Accessed 2026-09-19.
22. Chen, K.; Huang, F.; Liu, Z.; Yu, H.; Meng, L.; Mo, S.; Zhang, L.; Song, Y. ACcoding: A graph-based dataset for online judge programming. *Scientific Data* **2024**, *11*. See also Author Correction, doi:10.1038/s41597-024-03693-3, <https://doi.org/10.1038/s41597-024-03392-z>.
23. Mozilla. Taskcluster Documentation: Queue Service Reference. <https://docs.taskcluster.net/docs/reference/platform/queue>. Accessed 2026-09-19.
24. Mozilla. Treeherder REST API Documentation. <https://treeherder.mozilla.org/docs/>. Accessed 2026-09-19.
25. Parallel Workloads Archive. The Intel Netbatch Logs. https://www.cs.huji.ac.il/labs/parallel/workload/1_intel_netbatch/, 2012.
26. Shai, O.; Shmueli, E.; Feitelson, D.G. Heuristics for Resource Matching in Intel's Compute Farm. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2013). Springer, 2014, Vol. 8429, *Lecture Notes in Computer Science*, pp. 116–135. https://doi.org/10.1007/978-3-662-43779-7_7.
27. Mitzenmacher, M. Scheduling with Predictions and the Price of Misprediction. In Proceedings of the 11th Innovations in Theoretical Computer Science Conference (ITCS 2020). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, Vol. 151, *LIPICs*, pp. 14:1–14:18. Preprint arXiv:1902.00732, <https://doi.org/10.4230/LIPICs.ITCS.2020.14>.
28. Azar, Y.; Leonardi, S.; Tuitou, N. Flow Time Scheduling with Uncertain Processing Time. In Proceedings of the 53rd Annual ACM SIGACT Symposium on Theory of Computing (STOC), 2021, pp. 1070–1080. <https://doi.org/10.1145/3406325.3451023>.
29. Azar, Y.; Leonardi, S.; Tuitou, N. Distortion-Oblivious Algorithms for Minimizing Flow Time. In Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), 2022, pp. 252–274. Preprint arXiv:2109.08424, <https://doi.org/10.1137/1.9781611977073.13>.
30. Dong, J.; Ibrahim, R. Shortest-Job-First Scheduling in Many-Server Queues with Impatient Customers and Noisy Service-Time Estimates. *Operations Research* **2025**, *73*, 3139–3155. <https://doi.org/10.1287/opre.2022.310>.

31. Shahout, R.; Mitzenmacher, M. SkipPredict: When to Invest in Predictions for Scheduling. In Proceedings of the Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024, pp. 105663–105701. Preprint arXiv:2402.03564, <https://doi.org/10.52202/079017-3352>.
32. Schwiegelshohn, U.; Yahyapour, R. Fairness in Parallel Job Scheduling. *Journal of Scheduling* **2000**, *3*, 297–320. [https://doi.org/10.1002/1099-1425\(200009/10\)3:5<297::AID-JOS50>3.0.CO;2-D](https://doi.org/10.1002/1099-1425(200009/10)3:5<297::AID-JOS50>3.0.CO;2-D).
33. Sabin, G.; Kochhar, G.; Sadayappan, P. Job Fairness in Non-Preemptive Job Scheduling. In Proceedings of the 2004 International Conference on Parallel Processing (ICPP 2004). IEEE Computer Society, 2004, pp. 186–194. <https://doi.org/10.1109/ICPP.2004.1327920>.
34. Sabin, G.; Sadayappan, P. Unfairness Metrics for Space-Sharing Parallel Job Schedulers. In Proceedings of the Job Scheduling Strategies for Parallel Processing (JSSPP 2005), 2005, Vol. 3834, *Lecture Notes in Computer Science*, pp. 238–256. https://doi.org/10.1007/11605300_12.
35. Mu'alem, A.W.; Feitelson, D.G. Utilization, predictability, workloads, and user runtime estimates in scheduling the IBM SP2 with backfilling. *IEEE Transactions on Parallel and Distributed Systems* **2001**, *12*, 529–543. <https://doi.org/10.1109/71.932708>.
36. Lindsay, A.M.; Galloway-Carson, M.; Johnson, C.R.; Bunde, D.P.; Leung, V.J. Backfilling with guarantees made as jobs arrive. *Concurrency and Computation: Practice and Experience* **2013**, *25*, 513–523. <https://doi.org/10.1002/cpe.2860>.
37. Talby, D.; Feitelson, D.G. Supporting Priorities and Improving Utilization of the IBM SP Scheduler Using Slack-Based Backfilling. In Proceedings of the 13th International Parallel Processing Symposium and 10th Symposium on Parallel and Distributed Processing (IPPS/SPDP 1999), 1999, pp. 513–517. <https://doi.org/10.1109/IPPS.1999.760525>.
38. Lifka, D.A. The ANL/IBM SP Scheduling System. In Proceedings of the Job Scheduling Strategies for Parallel Processing (IPPS '95 Workshop). Springer, 1995, Vol. 949, *Lecture Notes in Computer Science*, pp. 295–303. https://doi.org/10.1007/3-540-60153-8_35.
39. Yuan, Y.; Yang, G.; Wu, Y.; Zheng, W. PV-EASY: A Strict Fairness Guaranteed and Prediction Enabled Scheduler in Parallel Job Scheduling. In Proceedings of the 19th ACM International Symposium on High Performance Distributed Computing (HPDC '10), 2010, pp. 240–251. <https://doi.org/10.1145/1851476.1851505>.
40. Grosz, I.; Yang, K.; Scully, Z.; Harchol-Balter, M. Nudge: Stochastically Improving upon FCFS. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2021**, *5*, 1–29. Preprint arXiv:2106.01492, <https://doi.org/10.1145/3460088>.
41. Van Houdt, B. On the Stochastic and Asymptotic Improvement of First-Come First-Served and Nudge Scheduling. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2022**, *6*, 1–22. Preprint arXiv:2206.10428, <https://doi.org/10.1145/3570610>.
42. Charlet, N.; Van Houdt, B. Tail Optimality and Performance Analysis of the Nudge*(M) Scheduling Algorithm. *Operations Research* **2026**, *74*, 2161–2178. <https://doi.org/10.1287/opre.2024.1061>.
43. Grosz, I.; Harchol-Balter, M.; Scheller-Wolf, A. WCFS: a new framework for analyzing multiserver systems. *Queueing Systems* **2022**, *102*, 143–174. Preprint arXiv:2109.12663, <https://doi.org/10.1007/s11134-022-09848-6>.
44. Grosz, I.; Scully, Z.; Harchol-Balter, M. Load Balancing Guardrails: Keeping Your Heavy Traffic on the Road to Low Response Times. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2019**, *3*, 1–31. Preprint arXiv:1905.03439, <https://doi.org/10.1145/3341617.3326157>.
45. Demers, A.; Keshav, S.; Shenker, S. Analysis and Simulation of a Fair Queueing Algorithm. In Proceedings of the Symposium Proceedings on Communications Architectures & Protocols (SIGCOMM '89). ACM, 1989, pp. 1–12. <https://doi.org/10.1145/75246.75248>.
46. Parekh, A.K.; Gallager, R.G. A generalized processor sharing approach to flow control in integrated services networks: the single-node case. *IEEE/ACM Transactions on Networking* **1993**, *1*, 344–357. <https://doi.org/10.1109/90.234856>.
47. Bennett, J.C.R.; Zhang, H. WF²Q: Worst-case Fair Weighted Fair Queueing. In Proceedings of the IEEE INFOCOM '96, 1996, Vol. 1, pp. 120–128. <https://doi.org/10.1109/INFCOM.1996.497885>.
48. Stoica, I.; Abdel-Wahab, H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation. Technical Report TR-95-22, Department of Computer Science, Old Dominion University, Norfolk, Virginia, 1995. Revised 26 January 1996.
49. Graham, R.L. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal on Applied Mathematics* **1969**, *17*, 416–429. <https://doi.org/10.1137/0117039>.
50. Tsafir, D.; Etsion, Y.; Feitelson, D.G. Backfilling Using System-Generated Predictions Rather than User Runtime Estimates. *IEEE Transactions on Parallel and Distributed Systems* **2007**, *18*, 789–803. <https://doi.org/10.1109/TPDS.2007.70606>.
51. Gaussier, E.; Glesser, D.; Reis, V.; Trystram, D. Improving backfilling by using machine learning to predict running times. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '15), 2015, pp. 1–10. <https://doi.org/10.1145/2807591.2807646>.
52. Gaussier, E.; Lelong, J.; Reis, V.; Trystram, D. Online Tuning of EASY-Backfilling using Queue Reordering Policies. *IEEE Transactions on Parallel and Distributed Systems* **2018**, *29*, 2304–2316. <https://doi.org/10.1109/TPDS.2018.2820699>.

53. Zhang, E.; Wu, F.; Lu, X. An Intelligent Scheduling System for Large-Scale Online Judging. In Proceedings of the Computer Science and Education. Computer Science and Technology (ICCSE 2023). Springer Nature Singapore, 2024, Vol. 2023, *Communications in Computer and Information Science*, pp. 265–279. https://doi.org/10.1007/978-981-97-0730-0_24.
54. Fu, Y.; Zhu, S.; Su, R.; Qiao, A.; Stoica, I.; Zhang, H. Efficient LLM Scheduling by Learning to Rank. In Proceedings of the Advances in Neural Information Processing Systems 37 (NeurIPS 2024), 2024, pp. 59006–59029. Preprint arXiv:2408.15792, <https://doi.org/10.52202/079017-1882>.
55. Li, Y.; Chen, Y.; Chen, J.; Choukse, E.; Qiu, H.; Suh, G.E.; Fonseca, R.; Scully, Z.; Gupta, U. Beyond Prediction: Tail-Aware Scheduling for LLM Inference. In Proceedings of the 43rd International Conference on Machine Learning (ICML 2026), Seoul, South Korea, 2026; Vol. 306, *Proceedings of Machine Learning Research*. Preprint arXiv:2606.18431.
56. Wu, B.; Zhong, Y.; Zhang, Z.; Liu, S.; Liu, F.; Sun, Y.; Huang, G.; Liu, X.; Jin, X. Fast Distributed Inference Serving for Large Language Models, 2023, [arXiv:cs.LG/2305.05920]. <https://doi.org/10.48550/arXiv.2305.05920>.
57. Luo, M.; Shi, X.; Cai, C.; Zhang, T.; Wong, J.; Wang, Y.; Wang, C.; Huang, Y.; Chen, Z.; Gonzalez, J.E.; et al. Autellix: An Efficient Serving Engine for LLM Agents as General Programs, 2025, [arXiv:cs.LG/2502.13965]. <https://doi.org/10.48550/arXiv.2502.13965>.
58. Fu, Y.; Liu, L.; Wang, H.; Cheng, Y.; Chen, S. SFS: Smart OS Scheduling for Serverless Functions. In Proceedings of the SC22: International Conference for High Performance Computing, Networking, Storage and Analysis. IEEE, 2022, pp. 1–16. Preprint arXiv:2209.01709, <https://doi.org/10.1109/SC41404.2022.00047>.
59. Fu, Y.; Shi, R.; Wang, H.; Chen, S.; Cheng, Y. ALPS: An Adaptive Learning, Priority OS Scheduler for Serverless Functions. In Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC 24). USENIX Association, 2024, pp. 19–36.
60. Massey, W.A.; Whitt, W. Peak congestion in multi-server service systems with slowly varying arrival rates. *Queueing Systems* **1997**, *25*, 157–172. <https://doi.org/10.1023/A:1019156418862>.
61. Jennings, O.B.; Mandelbaum, A.; Massey, W.A.; Whitt, W. Server Staffing to Meet Time-Varying Demand. *Management Science* **1996**, *42*, 1383–1394. <https://doi.org/10.1287/mnsc.42.10.1383>.
62. Robinson, J.; Ranjan, R.; Hu, W.; Huang, K.; Han, J.; Dobles, A.; Fey, M.; Lenssen, J.E.; Yuan, Y.; Zhang, Z.; et al. RelBench: A Benchmark for Deep Learning on Relational Databases. In Proceedings of the Advances in Neural Information Processing Systems 37, Datasets and Benchmarks Track (NeurIPS 2024), 2024. Preprint arXiv:2407.20060.
63. Grinsztajn, L.; Oyallon, E.; Varoquaux, G. Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? In Proceedings of the Advances in Neural Information Processing Systems 35 (NeurIPS 2022), 2022, pp. 507–520. <https://doi.org/10.52202/068431-0037>.
64. Xu, K.; Hu, W.; Leskovec, J.; Jegelka, S. How Powerful are Graph Neural Networks? In Proceedings of the International Conference on Learning Representations (ICLR 2019), 2019. Preprint arXiv:1810.00826.
65. Harchol-Balter, M. *Performance Modeling and Design of Computer Systems: Queueing Theory in Action*; Cambridge University Press, 2013. <https://doi.org/10.1017/CBO9781139226424>.
66. Scully, Z.; Harchol-Balter, M.; Scheller-Wolf, A. SOAP: One Clean Analysis of All Age-Based Scheduling Policies. *Proceedings of the ACM on Measurement and Analysis of Computing Systems* **2018**, *2*, 1–30. Preprint arXiv:1712.00790, <https://doi.org/10.1145/3179419>.
67. Jørgensen, B. Exponential Dispersion Models. *Journal of the Royal Statistical Society, Series B* **1987**, *49*, 127–145. <https://doi.org/10.1111/j.2517-6161.1987.tb01685.x>.
68. Yang, Y.; Qian, W.; Zou, H. Insurance Premium Prediction via Gradient Tree-Boosted Tweedie Compound Poisson Models. *Journal of Business & Economic Statistics* **2018**, *36*, 456–470. <https://doi.org/10.1080/07350015.2016.1200981>.
69. Schrage, L.E.; Miller, L.W. The Queue M/G/1 with the Shortest Remaining Processing Time Discipline. *Operations Research* **1966**, *14*, 670–684. <https://doi.org/10.1287/opre.14.4.670>.
70. Grosz, I.; Scully, Z.; Harchol-Balter, M. SRPT for multiserver systems. *Performance Evaluation* **2018**, 127–128, 154–175. Preprint arXiv:1805.07686, <https://doi.org/10.1016/j.peva.2018.10.001>.
71. Daley, D.J. Certain optimality properties of the first-come first-served discipline for G/G/s queues. *Stochastic Processes and their Applications* **1987**, *25*, 301–308. [https://doi.org/10.1016/0304-4149\(87\)90208-0](https://doi.org/10.1016/0304-4149(87)90208-0).
72. Cruz, R.L. A Calculus for Network Delay, Part I: Network Elements in Isolation. *IEEE Transactions on Information Theory* **1991**, *37*, 114–131. <https://doi.org/10.1109/18.61109>.
73. Le Boudec, J.Y.; Thiran, P. *Network Calculus: A Theory of Deterministic Queueing Systems for the Internet*; Vol. 2050, *Lecture Notes in Computer Science*, Springer: Berlin, Heidelberg, 2001. <https://doi.org/10.1007/3-540-45318-0>.
74. Wolff, R.W. Upper bounds on work in system for multichannel queues. *Journal of Applied Probability* **1987**, *24*, 547–551. Also indexed as doi:10.2307/3214279, <https://doi.org/10.1017/S0021900200031193>.
75. Smith, W.E. Various optimizers for single-stage production. *Naval Research Logistics Quarterly* **1956**, *3*, 59–66. <https://doi.org/10.1002/nav.3800030106>.

76. Kuzilek, J.; Hlosta, M.; Zdrahal, Z. Open University Learning Analytics dataset. *Scientific Data* **2017**, *4*, 170171. <https://doi.org/10.1038/sdata.2017.171>.
77. Universidade Federal do Amazonas, Instituto de Computação. CodeBench. <https://codebench.icomp.ufam.edu.br/>. Accessed 2026-09-23.
78. Feitelson, D.G. Resampling with Feedback: A New Paradigm of Using Workload Data for Performance Evaluation. In *Job Scheduling Strategies for Parallel Processing (JSSPP 2021)*; Springer, 2021; Vol. 12985, *Lecture Notes in Computer Science*, pp. 3–32. https://doi.org/10.1007/978-3-030-88224-2_1.
79. Shmueli, E.; Feitelson, D.G. On Simulation and Design of Parallel-Systems Schedulers: Are We Doing the Right Thing? *IEEE Transactions on Parallel and Distributed Systems* **2009**, *20*, 983–996. <https://doi.org/10.1109/TPDS.2008.152>.
80. Smith, D.R.; Whitt, W. Resource Sharing for Efficiency in Traffic Systems. *Bell System Technical Journal* **1981**, *60*, 39–55. <https://doi.org/10.1002/j.1538-7305.1981.tb00221.x>.
81. Harchol-Balter, M.; Schroeder, B.; Bansal, N.; Agrawal, M. Size-based scheduling to improve web performance. *ACM Transactions on Computer Systems* **2003**, *21*, 207–233. <https://doi.org/10.1145/762483.762486>.
82. Harchol-Balter, M. Task assignment with unknown duration. *Journal of the ACM* **2002**, *49*, 260–288. Conference version: Proceedings of the 20th International Conference on Distributed Computing Systems (ICDCS 2000), pp. 214–224, doi:10.1109/ICDCS.2000.840932, <https://doi.org/10.1145/506147.506154>.
83. Parallel Workloads Archive. The LPC Log. https://www.cs.huji.ac.il/labs/parallel/workload/l_lpc/. Parallel Workloads Archive; data provided by Emmanuel Medernach; accessed 2026-09-23.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.